

SAPIENZA
UNIVERSITÀ DI ROMA

Distributed Systems

Appunti del corso

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Corso di Laurea Magistrale in Computer Science

Docente
Prof. Alessandro Mei

Appunti di
Alessio Cavedal
GitHub: https://github.com/AlessioCavedal

Anno Accademico 2026/2027

Indice

1 Sistemi distribuiti e computazioni **1**
1.1 Il problema: valutare un predicato globale 1
1.2 Il modello: sistemi distribuiti asincroni 3
1.3 Computazioni distribuite 4

2 Stati globali, tagli e consistenza **7**
2.1 Stati globali, tagli e run 7
2.2 Monitoraggio attivo e deadlock fantasma 9
2.3 Tagli e run consistenti 10
2.4 Il reticolo degli stati globali 12

3 Osservare una computazione: orologio reale e clock logici **15**
3.1 Monitoraggio passivo e osservazioni 15
3.2 Regole di consegna e consegna FIFO 16
3.3 Orologio reale e clock condition 17
3.4 Clock logici di Lamport 18
3.5 Consegnare le notifiche in ordine di clock logico 19

4 Consegna causale e vector clock **21**
4.1 Consegna causale 21
4.2 La strong clock condition 22
4.3 Storie causali 23
4.4 Vector clock 23
4.5 Confrontare i vector clock 25
4.6 Consegna causale con i vector clock 26

5 Snapshot distribuiti **29**
5.1 Che cosa registra uno snapshot 29
5.2 Primo protocollo: un orologio reale 29
5.3 Dall'orologio reale ai marker 30
5.4 Il protocollo di Chandy–Lamport 31
5.5 Perché lo stato è consistente 32
5.6 Proprietà dello snapshot 33

6 Guasti e commit atomico **35**
6.1 Transazioni distribuite 35
6.2 Il modello dei guasti 36
6.3 Il problema del commit atomico 37
6.4 Incertezza e blocco 38

7 Il protocollo Two-Phase Commit **40**
7.1 Il protocollo 40
7.2 Timeout e protocollo di terminazione 41
7.3 Recupero e DT log 43
7.4 Valutazione del 2PC 46

Capitolo 1

Sistemi distribuiti e computazioni

Molti problemi dei sistemi distribuiti hanno la stessa forma: bisogna accorgersi che lo stato complessivo del sistema soddisfa una certa condizione e, quando succede, reagire. Per affrontarli serve prima un modello preciso di che cosa sia un sistema distribuito e di come si svolga una computazione al suo interno. Questo capitolo fissa il modello usato nei capitoli seguenti: processi che comunicano solo con messaggi, senza un orologio comune, e un ordine tra gli eventi basato soltanto su cause ed effetti.

1.1 Il problema: valutare un predicato globale

Ecco alcuni problemi tipici dei sistemi distribuiti:

- rilevare un *deadlock*, cioè un gruppo di processi bloccati, ciascuno in attesa di un altro;
- capire se una computazione distribuita è terminata;
- accorgersi che il *token* di un protocollo a gettone è andato perso;
- trovare gli oggetti che nessun processo può più raggiungere, per liberarne la memoria (*garbage collection* distribuita);
- salvare uno stato da cui ripartire dopo un guasto (*checkpointing*);
- riconfigurare un programma mentre è in esecuzione, per esempio per bilanciare il carico;
- fare *debugging* e monitoraggio di un'applicazione distribuita.

In tutti questi casi bisogna controllare una condizione sullo stato complessivo del sistema, e una condizione così si esprime con un predicato.

Un **predicato** è un'affermazione sullo stato del sistema che può essere vera o falsa, come «la computazione è finita». **Valutarlo** vuol dire stabilire se, in un certo stato, è vera o falsa. Quanto sia facile farlo dipende da quali informazioni servono. Un **predicato locale** dipende solo dallo stato di un processo: «p_1 sta aspettando una risposta» è vero o falso a seconda delle variabili di p_1, e p_1 lo può valutare da

solo. Un **predicato globale** dipende invece dagli stati di più processi insieme, e spesso anche dai messaggi in viaggio tra loro: nessun processo lo può valutare da solo, perché ognuno conosce soltanto il proprio stato.

Le condizioni dei problemi elencati sono predicati globali, che si indicano con Φ (la lettera greca *phi* maiuscola). Per esempio:

- *deadlock*: Φ = «c'è un ciclo di processi, ciascuno in attesa del successivo». p_1 sa di aspettare p_2, ma non sa se p_2 aspetta p_3 e p_3 aspetta p_1: il ciclo si vede solo mettendo insieme gli stati di tutti;
- terminazione: Φ = «tutti i processi sono inattivi e non ci sono messaggi in viaggio». Non basta che ogni processo sia inattivo, perché un processo inattivo che riceve un messaggio si rimette al lavoro;
- *token* perso: Φ = «nessun processo ha il *token* e nessun messaggio in viaggio lo trasporta».

Definizione (Global Predicate Evaluation). La **Global Predicate Evaluation** (GPE, valutazione di un predicato globale) è il problema di stabilire se lo **stato globale** del sistema, cioè l'insieme degli stati dei singoli processi, soddisfa un predicato globale Φ.

Se ne occupa un processo, indicato con p_0, che dal capitolo 2 si chiamerà monitor. GPE è un problema generale perché non dipende da quale Φ si sceglie: un metodo che lo risolve è il cuore della soluzione di tutti i problemi elencati, e per ciascuno resta da scegliere il predicato giusto e la reazione da eseguire quando è vero, per esempio interrompere un processo del ciclo o rigenerare il *token*.

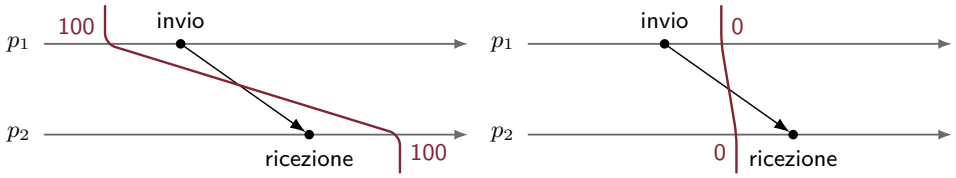
Su un solo computer valutare un predicato è facile: le variabili da cui dipende stanno nella stessa memoria e si leggono tutte nello stesso momento. In un sistema distribuito i processi non condividono memoria e comunicano solo scambiandosi messaggi. Un processo che vuole conoscere lo stato globale deve farsi mandare gli stati degli altri, e i messaggi impiegano tempi variabili e imprevedibili: ogni stato viene letto in un momento diverso, e intanto il sistema va avanti.

Esempio (un bonifico). Due processi gestiscono due conti bancari: p_1 ha 100 euro e p_2 nessuno. p_1 fa a p_2 un bonifico di 100 euro: toglie la somma dal proprio saldo e la invia con un messaggio, e p_2 la aggiunge al suo quando il messaggio arriva. Il denaro non si crea e non si distrugge, quindi i due saldi, più il denaro in viaggio, fanno sempre 100 euro. Il processo p_0 controlla che sia così e valuta Φ = «il denaro complessivo non è 100 euro», cioè «c'è un errore»: chiede il saldo a p_1 e a p_2 e somma le risposte. Le richieste arrivano ai due processi in momenti diversi, e la Figura 1.1 mostra due casi.

(a) La richiesta arriva a p_1 *prima* che invii il bonifico, e p_1 risponde 100; arriva a p_2 *dopo* che l'ha ricevuto, e anche p_2 risponde 100. La somma è 200 euro e p_0 segnala un errore che non c'è mai stato. Lo stato ricostruito è impossibile: p_2 ha ricevuto un bonifico che p_1 non ha ancora inviato.

(b) La richiesta arriva a p_1 *dopo* l'invio, e p_1 risponde 0; arriva a p_2 *prima*

della ricezione, e p_2 risponde 0. Anche qui p_0 segnala un errore, ma per un motivo diverso: lo stato è possibile, e i 100 euro sono nel messaggio in viaggio, che nessuna delle due risposte comprende. Per non sbagliare p_0 deve contare anche i messaggi in viaggio, per esempio facendosi dire da p_1 i bonifici inviati e da p_2 quelli ricevuti.



(a) 100 + 100 = 200 (b) 0 + 0 = 0, e 100 in viaggio

Figura 1.1. Il bonifico dell'esempio: il tempo scorre da sinistra a destra, e la freccia è il messaggio con i 100 euro. Le linee rosse passano per i punti in cui le richieste di p_0 arrivano a p_1 e a p_2; accanto, in rosso, i saldi comunicati.

In generale lo stato che un processo ricostruisce in questo modo può essere:

- *obsoleto*: quando le risposte arrivano, il sistema è già andato avanti, e nel caso (b) il bonifico potrebbe essere già arrivato;
- *incompleto*: alcune informazioni sono ancora in viaggio, come i 100 euro nel caso (b);
- *inconsistente*: mette insieme pezzi presi in momenti incompatibili e descrive una situazione in cui il sistema non si sarebbe mai potuto trovare, come i 200 euro nel caso (a).

Comunicare più spesso rende la vista più aggiornata e più completa, ma non basta a renderla consistente: bisogna ragionare sull'ordine in cui le informazioni vengono osservate e su che cosa contengono. Risolvere GPE vuol dire quindi costruire uno stato globale *consistente*, cioè uno in cui il sistema si sarebbe potuto trovare davvero, e valutare Φ su quello. Il capitolo 2 definisce con precisione la consistenza. I capitoli successivi mostrano due modi di costruire stati globali consistenti: in uno un processo *osserva* passivamente il sistema (capitoli 3 e 4), nell'altro lo *interroga* attivamente (capitolo 5).

1.2 Il modello: sistemi distribuiti asincroni

Definizione (sistema distribuito). Un **sistema distribuito** è un insieme di processi sequenziali $p_1, \ldots, p_n$ e una rete che realizza **canali** di comunicazione unidirezionali tra coppie di processi. Nel nostro modello:

- i canali sono **affidabili**: ogni messaggio inviato prima o poi arriva;
- i canali **non sono FIFO**: possono consegnare i messaggi in un ordine diverso da quello di invio;

- la rete è **fortemente connessa**: ogni processo può comunicare con ogni altro, eventualmente passando per processi intermedi, anche se non c'è un canale diretto tra ogni coppia.

Quando si sceglie un modello conviene fare meno ipotesi possibile. Le due grandezze su cui si può contare o no sono la velocità relativa dei processi e il ritardo dei messaggi.

Definizione (sistema asincrono). Un sistema distribuito è **asincrono** se non esiste alcun limite né alle velocità relative dei processi né ai ritardi dei messaggi. Se invece questi limiti esistono e sono noti, il sistema è **sincrono**.

In un sistema asincrono un processo non può contare su orologi sincronizzati con quelli degli altri, né ragionare in termini di un tempo reale comune: l'unico strumento di sincronizzazione è la comunicazione. Ci sono due buoni motivi per studiare proprio questo modello.

- *È il più debole.* Fa meno ipotesi di qualunque altro, quindi una soluzione che funziona in un sistema asincrono funziona in qualunque sistema distribuito, e il suo costo è un limite superiore per tutti gli altri. Non vale il contrario: un algoritmo pensato per un sistema sincrono può sfruttare i limiti di tempo, per esempio concludendo che un processo non ha inviato nulla perché entro un certo tempo non è arrivato niente, ma in un sistema asincrono questo ragionamento è sbagliato, perché un ritardo lungo è sempre possibile.
- *È realistico.* I componenti fisici sono spesso sincroni (le velocità dei processori e i ritardi sui collegamenti sono limitati), ma gli strati di software che li condividono tra molti processi, come *scheduling*, code e ritrasmissioni, rendono imprevedibili i tempi visti dalle applicazioni. Il sistema che ne risulta si descrive meglio come asincrono.

Il teorema CAP. Per i sistemi che conservano copie degli stessi dati su più nodi, il **teorema CAP** dice che non si possono garantire insieme *Consistency* (tutti i nodi vedono gli stessi dati), *Availability* (ogni richiesta a un nodo funzionante riceve risposta) e *Partition tolerance* (il sistema funziona anche se la rete si divide), ma al più due delle tre. Poiché le partizioni possono sempre capitare, in pratica si sceglie tra consistenza e disponibilità. La *consistency* di CAP riguarda le copie dei dati, e non c'entra con la consistenza dei tagli del capitolo 2.

1.3 Computazioni distribuite

Una **computazione distribuita** è l'esecuzione di un programma distribuito da parte dei processi del sistema. L'attività di ogni processo, che è sequenziale, si descrive come una sequenza di **eventi**. Un evento può essere:

- **interno**: cambia soltanto lo stato locale del processo;

- un **invio** $send(m)$: il processo mette il messaggio m sul canale verso il destinatario;
- una **ricezione** $receive(m)$: il processo preleva m dal canale in arrivo.

Ogni messaggio ha un identificatore unico (per esempio l'indice del mittente e un numero di sequenza), così ogni ricezione corrisponde esattamente a un invio; e qualunque forma di comunicazione di più alto livello, come le chiamate di procedura remota o il *broadcast*, si riduce a coppie di invii e ricezioni.

Definizione (storia locale e storia globale). La **storia locale** di p_i è la sequenza, eventualmente infinita, dei suoi eventi

$$h_i = e_i^1 \, e_i^2 \, e_i^3 \ldots$$

numerati nell'ordine in cui p_i li esegue: è l'**enumerazione canonica**. Con $h_i^k = e_i^1 \ldots e_i^k$ si indica il prefisso dei primi k eventi (h_i^0 è la sequenza vuota). La **storia globale** è l'insieme di tutti gli eventi della computazione: $H = h_1 \cup \cdots \cup h_n$.

1.3.1 Precedenza causale

La storia globale dice quali eventi sono avvenuti, ma non *quando* gli eventi di processi diversi sono avvenuti l'uno rispetto all'altro: in un sistema asincrono non c'è un tempo comune a cui riferirsi. L'unico ordine che ha senso è quello di causa ed effetto: un evento deve venire prima di un altro solo se può averlo influenzato, cioè se c'è stato un flusso di informazione dal primo al secondo.

Definizione (precedenza causale). La **precedenza causale** $\to$ (*causal precedence*) è la relazione tra gli eventi di H definita da tre regole:

1. se e ed e' sono eventi dello stesso processo ed e viene prima di e', allora $e \to e'$;
2. se $e = send(m)$ ed $e' = receive(m)$, allora $e \to e'$;
3. se $e \to e'$ ed $e' \to e''$, allora $e \to e''$ (transitività).

Due eventi sono **concorrenti**, e si scrive $e \parallel e'$, se non vale né $e \to e'$ né $e' \to e$.

$e \to e'$ si legge «e precede causalmente e'»: l'esecuzione di e, e il suo risultato, *possono* aver influenzato e'. Solo per una coppia invio–ricezione l'influenza è certa; in generale si può dire soltanto che è possibile. Lamport la chiamava *happened before*, ma il nome inganna: conta la possibilità di un'influenza, non l'ordine nel tempo reale. Due eventi concorrenti, invece, non possono essersi influenzati in nessun verso, e il loro ordine nel tempo reale è irrilevante.

Una computazione distribuita, quindi, non è una sequenza di eventi ma un insieme parzialmente ordinato (*poset*): formalmente è la coppia $(H, \to)$. L'ordine è *parziale* perché le coppie di eventi concorrenti non sono ordinate.

1.3.2 Diagrammi spazio-tempo

Una computazione si rappresenta comodamente con un **diagramma spazio-tempo** (*space-time diagram*) come quello della Figura 1.2. Ogni processo è una linea orizzontale, con il tempo che scorre da sinistra a destra; gli eventi sono punti sulla linea, e ogni messaggio è una freccia che parte dall'evento di invio e arriva all'evento di ricezione. Gli eventi interni non hanno frecce.

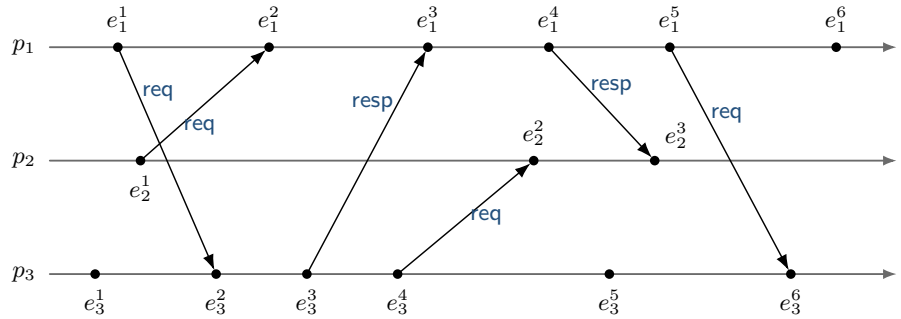


Figura 1.2. La computazione d'esempio: tre processi che si scambiano richieste (req) e risposte (resp).

Esempio (la computazione d'esempio). La computazione della Figura 1.2 tornerà in quasi tutti i capitoli. È un sistema client–server con chiamate di procedura remota: ogni processo può fare sia da client sia da server, e dopo aver inviato una richiesta (req) aspetta la risposta (resp). Per esempio p_1 invia una richiesta a p_3 (e_1^1); p_3 la riceve (e_3^2) e risponde (e_3^3), e la risposta arriva a p_1 in e_1^3. Nel frattempo p_1 ha ricevuto una richiesta da p_2 (e_1^2), a cui risponde in e_1^4. Gli eventi e_3^1, e_3^5 e e_1^6 sono interni.

Il diagramma rende facile leggere la precedenza causale: $e \to e'$ se e solo se si può andare da e a e' con un cammino che si muove verso destra lungo le linee dei processi e segue le frecce nel loro verso. Se un cammino così non c'è in nessuno dei due versi, i due eventi sono concorrenti.

Esempio (leggere la precedenza causale). Nella Figura 1.2:

- $e_2^1 \to e_3^6$, grazie al cammino $e_2^1 \to e_1^2 \to e_1^3 \to e_1^4 \to e_1^5 \to e_3^6$: la richiesta di p_2 arriva a p_1, si prosegue lungo p_1, e da e_1^5 parte la richiesta che p_3 riceve in e_3^6;
- $e_2^2 \parallel e_3^6$: dopo e_2^2 il processo p_2 non invia più messaggi, quindi da e_2^2 non si arriva a p_3; e e_3^6 è l'ultimo evento di p_3, quindi da lì non si arriva da nessuna parte;
- $e_1^1 \parallel e_2^1$: sono i primi eventi dei rispettivi processi e nessun messaggio li collega.

Capitolo 2

Stati globali, tagli e consistenza

Per valutare un predicato globale bisogna prima ricostruire uno stato globale. Questo capitolo definisce stati locali e globali e i tagli che li individuano, e mostra con un esempio come un monitor ingenuo possa ricostruire uno stato mai esistito e trarne conclusioni sbagliate. La nozione che separa gli stati sensati da quelli impossibili è la consistenza; gli stati consistenti, ordinati, formano il reticolo degli stati globali.

2.1 Stati globali, tagli e run

Indichiamo con σ_i^k lo **stato locale** di p_i subito dopo l'evento e_i^k, e con σ_i^0 il suo stato iniziale, prima di qualunque evento. Lo stato locale comprende i valori delle variabili del processo e, se servono, le sequenze dei messaggi inviati e ricevuti su ciascun canale. Lo **stato globale** è la n-upla degli stati locali, uno per processo: $\Sigma = (\sigma_1, \ldots, \sigma_n)$.

Non serve aggiungere allo stato globale lo stato dei canali, cioè i messaggi in viaggio: se gli stati locali registrano i messaggi inviati e ricevuti, i messaggi in viaggio da p_i a p_j sono quelli che p_i ha inviato e p_j non ha ancora ricevuto. Gli stati dei canali torneranno in modo esplicito con gli snapshot (capitolo 5).

> **Storia e stato.** Storia e stato descrivono la stessa computazione, ma rispondono a domande diverse. La storia dice *che cosa è successo e in che ordine*: è la sequenza degli eventi, come l'elenco delle mosse di una partita a scacchi. Lo stato dice *com'è il processo in un certo punto*: sono i valori delle sue variabili, come la posizione sulla scacchiera.
> Per esempio, p_1 gestisce un conto che parte da 0 euro: riceve da p_2 un bonifico di 50 euro (e_1^1), deposita 100 euro (e_1^2) e ne preleva 30 (e_1^3).
>
> - «Il saldo supera i 100 euro?» Lo stato σ_1^3 lo dice subito: il saldo è 120 euro. Nella storia questo numero non c'è, e per trovarlo bisogna rifare i conti dall'inizio, cioè ricostruire lo stato.
> - «Il bonifico è arrivato prima del prelievo?» La storia lo dice subito. Lo stato no, perché a 120 euro si arriva in molti modi, anche con un solo deposito.

Lo stato σ_i^k corrisponde al prefisso h_i^k dei primi k eventi solo nel senso che ne è il risultato, come dopo k mosse c'è una sola posizione: dalla storia, partendo dallo stato iniziale, si ricava lo stato, ma non il contrario.
Per questo servono tutti e due. I predicati parlano di valori e si valutano sugli stati; l'ordine si studia sugli eventi, su cui si definiscono precedenza causale, clock e consistenza dei tagli. In GPE la storia dice *dove* si può guardare, cioè quali tagli sono consistenti, e lo stato dice *che cosa* si vede lì, cioè i valori su cui valutare il predicato globale Φ.
Le sequenze dei messaggi inviati e ricevuti sono un pezzo di storia annotato nello stato. Servono perché nessun processo vede la storia: ognuno ha solo il proprio stato e i messaggi che riceve, e ciò che gli serve sapere del passato deve averlo annotato. Annotare non trasforma però lo stato nella storia: anche se p_i annotasse tutti i propri eventi, σ_i^k conterrebbe solo una copia del passato fino a quel punto, mentre la storia descrive l'intera computazione vista da fuori ed è il riferimento con cui si giudica se ciò che i processi ricostruiscono è corretto. In pratica si annota solo il riassunto che serve, come il totale inviato e ricevuto su ogni canale, o i clock dei capitoli 3 e 4.

Per indicare *quale* stato globale si considera basta dire fino a che punto è arrivato ciascun processo. È l'idea di taglio.

Definizione (taglio). Un **taglio** (*cut*) è un sottoinsieme C della storia globale formato da un prefisso della storia locale di ogni processo:

$$C = h_1^{c_1} \cup \cdots \cup h_n^{c_n}.$$

Si indica con la tupla $(c_1, \ldots, c_n)$ degli indici dell'ultimo evento incluso per ciascun processo. Gli ultimi eventi inclusi, $(e_1^{c_1}, \ldots, e_n^{c_n})$, formano la **frontiera** del taglio, e lo stato globale che gli corrisponde è $(\sigma_1^{c_1}, \ldots, \sigma_n^{c_n})$.

Un taglio non è quindi un sottoinsieme qualunque della storia globale: di ogni processo p_i prende i primi c_i eventi, senza buchi. Non può contenere, per esempio, e_1^3 senza e_1^1 ed e_1^2.

Nel diagramma spazio-tempo un taglio è una linea che attraversa dall'alto in basso tutti i processi: gli eventi a sinistra della linea sono nel taglio, quelli a destra no. La Figura 2.1 mostra due tagli della computazione d'esempio, $C = (5, 2, 4)$ e $C' = (3, 2, 6)$.

Un taglio dice quali eventi sono avvenuti, non in che ordine. Per ragionare sulle esecuzioni serve anche un ordine totale: in un'esecuzione reale tutti gli eventi, anche di processi diversi, avvengono in un qualche ordine. Due eventi simultanei si possono ordinare in modo arbitrario, per esempio mettendo prima quello del processo con indice minore.

Definizione (run). Un **run** (esecuzione) di una computazione è un ordine totale di tutti gli eventi della storia globale che rispetta ogni storia locale: gli eventi di ciascun processo p_i compaiono nel run nello stesso ordine in cui compaiono

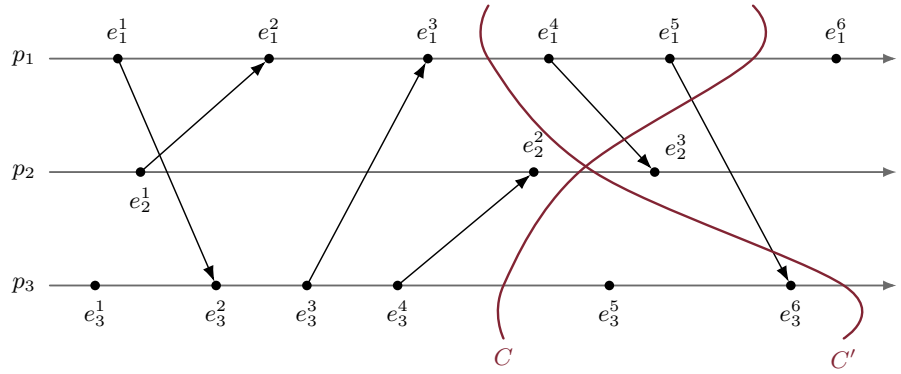


Figura 2.1. Due tagli della computazione d'esempio: $C = (5, 2, 4)$ e $C' = (3, 2, 6)$.

in h_i.

Una computazione ha in generale molti run, e un run non corrisponde per forza a un'esecuzione possibile: la definizione non impedisce, per esempio, di mettere la ricezione di un messaggio prima del suo invio. A questo rimediano i run consistenti della Sezione 2.3.

2.2 Monitoraggio attivo e deadlock fantasma

Supponiamo che un solo processo, il **monitor** p_0, abbia il compito di valutare il predicato Φ. Può essere uno dei processi $p_1, \ldots, p_n$ oppure un processo esterno alla computazione, ma pur sempre parte del sistema e soggetto ai suoi ritardi. Gli eventi eseguiti per il monitoraggio non fanno parte della computazione e non cambiano la numerazione dei suoi eventi.

La strategia più semplice è attiva: p_0 chiede lo stato a tutti. Invia a ogni processo una richiesta; quando la riceve, p_i risponde con il proprio stato locale corrente; raccolte tutte le risposte, p_0 compone lo stato globale. I punti in cui le richieste arrivano ai processi definiscono un taglio, e lo stato costruito è quello di quel taglio. Il problema è che le richieste arrivano in momenti scollegati tra loro, quindi il taglio che ne risulta può essere qualunque.

Esempio (deadlock fantasma). Nella computazione d'esempio (Figura 1.2) ogni processo, dopo aver inviato una richiesta, resta bloccato finché non riceve la risposta, quindi il sistema può andare in deadlock. Per rilevarlo, ogni server tiene nel proprio stato locale l'elenco dei client da cui ha ricevuto richieste a cui non ha ancora risposto. Con queste informazioni si costruisce il **grafo delle attese** (*waits-for graph*, WFG): i nodi sono i processi, e c'è un arco da p_i a p_j se p_j ha ricevuto una richiesta da p_i e non ha ancora risposto, cioè se p_i sta aspettando p_j. Un ciclo nel grafo è un deadlock, perché ogni processo del ciclo aspetta il successivo. Il predicato da valutare è quindi Φ = «il WFG contiene un ciclo».
Supponiamo che le richieste di p_0 arrivino ai processi nei punti del taglio

$C' = (3, 2, 6)$ della Figura 2.1, e che p_0 riceva quindi σ_1^3, σ_2^2 e σ_3^6:

- in σ_3^6, p_3 ha ricevuto da p_1 la richiesta inviata in e_1^5 e non ha risposto: arco $p_1 \to p_3$;
- in σ_1^3, p_1 ha ricevuto la richiesta di p_2 (e_1^2) e non ha ancora risposto: arco $p_2 \to p_1$;
- in σ_2^2, p_2 ha ricevuto la richiesta di p_3 (e_2^2) e non ha risposto: arco $p_3 \to p_2$.

Il grafo contiene il ciclo $p_1 \to p_3 \to p_2 \to p_1$ (Figura 2.2) e p_0 dichiara un deadlock. Ma un osservatore esterno che vedesse l'intera computazione direbbe che il sistema non è mai in deadlock: p_1 risponde a p_2 (e_1^4) *prima* di inviare la nuova richiesta a p_3 (e_1^5). Il deadlock rilevato è un **deadlock fantasma** (*ghost deadlock*).

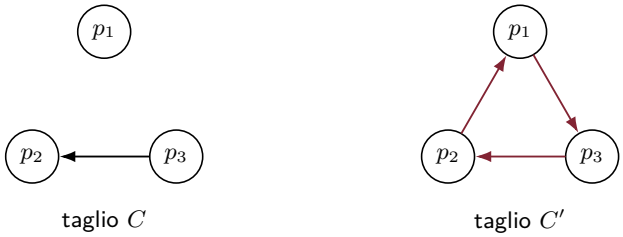


Figura 2.2. Il grafo delle attese costruito sullo stato del taglio C e su quello di C': solo il secondo contiene un ciclo.

Dov'è l'errore? Nello stato di C', p_3 risulta aver ricevuto una richiesta (e_3^6) che p_1 non ha ancora inviato (e_1^5 è fuori dal taglio): uno stato così non avrebbe mai potuto verificarsi. Se invece le richieste di p_0 arrivassero nei punti del taglio $C = (5, 2, 4)$, il grafo avrebbe il solo arco $p_3 \to p_2$ e nessun ciclo. Qui si vedono anche i ruoli di storia e stato: il grafo delle attese si costruisce dagli stati locali, ma l'errore si scopre guardando gli eventi.

2.3 Tagli e run consistenti

La precedenza causale è lo strumento giusto per distinguere i due tipi di taglio.

Definizione (taglio consistente). Un taglio C è **consistente** se, per ogni coppia di eventi e ed e',

$$(e \in C) \wedge (e' \to e) \;\Rightarrow\; e' \in C.$$

In parole: se un evento sta nel taglio, ci stanno anche tutti gli eventi che lo precedono causalmente, anche indirettamente. Un taglio consistente è «chiuso all'indietro» rispetto alla precedenza causale: non contiene un effetto senza le sue cause.

Dentro un singolo processo questa condizione è sempre soddisfatta, perché un taglio contiene prefissi delle storie locali. L'unico modo per violarla è un messaggio

ricevuto dentro il taglio ma *inviato* fuori. Nel diagramma il test è immediato: un taglio è consistente se ogni freccia che lo attraversa va da sinistra (dentro il taglio) verso destra (fuori); è inconsistente se una freccia lo attraversa da destra verso sinistra, cioè se c'è un messaggio ricevuto ma non ancora inviato. Nella Figura 2.1 il taglio C è consistente; C' no, perché la richiesta da e_1^5 a e_3^6 lo attraversa all'indietro.

Uno **stato globale consistente** è lo stato di un taglio consistente. La consistenza separa gli stati possibili da quelli impossibili.

- In uno stato inconsistente il sistema non si sarebbe mai potuto trovare: è il caso del deadlock fantasma e dei 200 euro del bonifico (Sezione 1.1).
- In uno stato consistente il sistema si sarebbe potuto trovare: un osservatore ideale, esterno al sistema, avrebbe potuto vederlo in qualche esecuzione. Non è detto, però, che ci si sia trovato davvero.

Il valore di un predicato ha quindi senso solo negli stati globali consistenti.

Esempio (uno stato possibile ma mai attraversato). Una computazione ha due soli eventi, e su p_1 ed f su p_2, senza messaggi tra loro: sono concorrenti. Supponiamo che in realtà e avvenga prima di f: l'esecuzione passa per lo stato in cui e è avvenuto ed f non ancora. Anche lo stato opposto, con f avvenuto ed e non ancora, è consistente: nessun messaggio lo vieta, e se f fosse stato un po' più veloce il sistema ci sarebbe passato. In realtà non ci è mai passato, ma nessuno dentro il sistema può accorgersene: senza un orologio comune, l'ordine reale di due eventi concorrenti non si osserva.

Nella Sezione 2.4 l'idea diventa generale: i run consistenti sono i cammini nel reticolo degli stati consistenti, e l'esecuzione reale è solo uno di essi. Lo snapshot del capitolo 5 costruirà proprio uno stato consistente che l'esecuzione reale può non aver attraversato (Sezione 5.6).

I tagli consistenti hanno un ruolo centrale. In un programma sequenziale un istante di tempo individua un punto preciso dell'esecuzione; in una computazione distribuita questo ruolo lo ha la frontiera di un taglio consistente. Anche «prima» e «dopo» vanno intesi rispetto ai tagli consistenti: un evento è prima di un taglio se sta a sinistra della sua frontiera, dopo se sta a destra.

La stessa idea si applica ai run.

Definizione (run consistente). Un run R è **consistente** se, per ogni coppia di eventi, $e \to e'$ implica che e compare prima di e' in R.

La condizione è diversa da quella dei tagli. Un taglio è un insieme, e chiede che le cause *ci siano*; un run contiene sempre tutti gli eventi, e chiede che le cause *vengano prima*. Le due cose si collegano così: un run è consistente esattamente quando, fermandolo in qualunque punto, si ottiene un taglio consistente. In pratica, siccome le storie locali sono già rispettate, basta che ogni invio compaia prima della propria ricezione.

Un run $R = e^1 e^2 \dots$ si può vedere anche come una sequenza di stati globali $\Sigma^0 \Sigma^1 \Sigma^2 \dots$, dove $\Sigma^0 = (\sigma_1^0, \dots, \sigma_n^0)$ è lo stato iniziale e ogni Σ^i si ottiene da Σ^{i-1}

eseguendo il singolo evento e^i; se il run è consistente, sono tutti consistenti. Due stati consecutivi sono legati dalla relazione *leads to* («porta a»), e Σ' è **raggiungibile** da Σ, in simboli $\Sigma \rightsquigarrow \Sigma'$, se esiste un run consistente che passa prima per Σ e poi per Σ'.

2.4 Il reticolo degli stati globali

L'insieme degli stati globali consistenti di una computazione, con la relazione *leads to*, forma un **reticolo** (*lattice*). È un modo di disegnare tutti gli stati in cui il sistema si può trovare e tutti i modi di passare dall'uno all'altro.

La notazione. Si scrive $\Sigma^{k_1 \cdots k_n}$ per lo stato globale $(\sigma_1^{k_1}, \ldots, \sigma_n^{k_n})$, quello in cui p_1 ha eseguito esattamente k_1 eventi, p_2 esattamente k_2, e così via: è lo stato del taglio $(k_1, \ldots, k_n)$. Con due processi si scrive Σ^{ab}, e le due cifre in apice sono due contatori separati, non un numero: Σ^{21} si legge «p_1 ha eseguito due eventi, p_2 uno», e Σ^{00} è lo stato iniziale. Il **livello** di uno stato è la somma $k_1 + \cdots + k_n$, cioè il numero totale di eventi eseguiti: Σ^{21} sta al livello 3.

> **Due notazioni da non confondere.** Nella Sezione 2.3 gli stati attraversati da un run sono scritti $\Sigma^0\Sigma^1\Sigma^2\ldots$: lì l'unico indice conta gli eventi eseguiti *lungo quel run*, di qualunque processo siano. In $\Sigma^{k_1 \cdots k_n}$, invece, c'è un indice per ogni processo. Per esempio, se un run comincia con $e_2^1\, e_1^1\, e_1^2$, il suo Σ^3 è Σ^{21}.

Come si costruisce. Prendiamo la computazione di due processi della Figura 2.3, a destra: p_1 ha sei eventi, p_2 cinque, e ci sono tre messaggi. Il reticolo si costruisce dall'alto.

1. In cima si mette lo stato iniziale Σ^{00}.
2. Da ogni stato si scende di un passo eseguendo un solo evento in più: il prossimo di p_1, e allora nella figura si scende a sinistra, da Σ^{ab} a $\Sigma^{a+1,b}$; oppure il prossimo di p_2, e si scende a destra, verso $\Sigma^{a,b+1}$.
3. Un passo è possibile solo se lo stato che si ottiene è consistente: un evento interno o un invio si può eseguire sempre, una ricezione solo se l'invio corrispondente è già stato eseguito.
4. Gli stati dello stesso livello stanno sulla stessa riga.

Con n processi il reticolo ha n assi, uno per processo, e già con tre processi è difficile da disegnare, ma la costruzione è la stessa.

La Figura 2.4 mostra i primi livelli, con l'evento eseguito a ogni passo. Da Σ^{00} si può eseguire e_1^1 oppure e_2^1. Da Σ^{10}, invece, non si può scendere a sinistra: il prossimo evento di p_1 è e_1^2, che riceve il messaggio inviato in e_2^1, non ancora eseguito. Lo stato Σ^{20}, quindi, non c'è. Da Σ^{11} lo stesso evento e_1^2 è possibile, perché ora e_2^1 è avvenuto, e si arriva a Σ^{21}.

Quali stati mancano lo dicono i messaggi, perché ognuno impone che la sua ricezione non avvenga prima dell'invio. Nella computazione della figura i vincoli su Σ^{ab} sono tre:

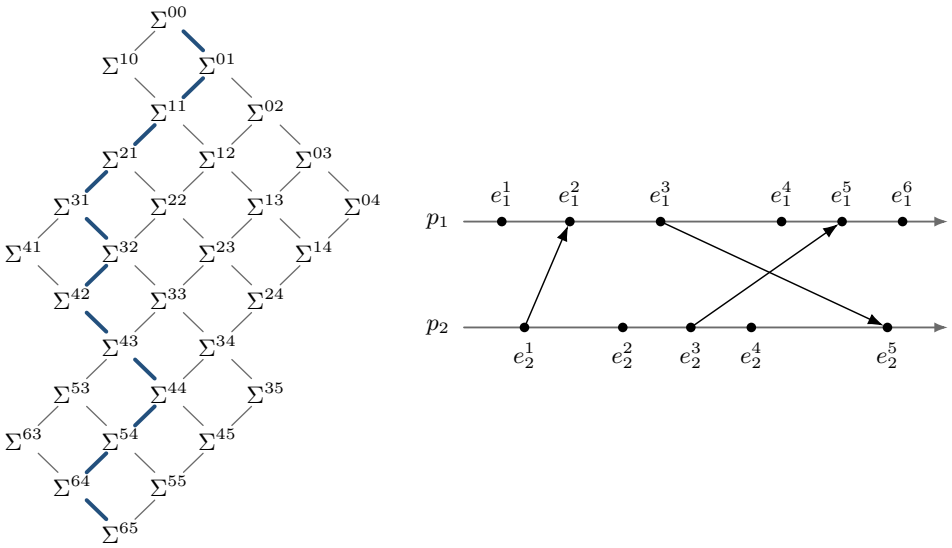


Figura 2.3. Una computazione di due processi (a destra) e il reticolo dei suoi stati globali consistenti (a sinistra). In blu un cammino, cioè un run consistente.

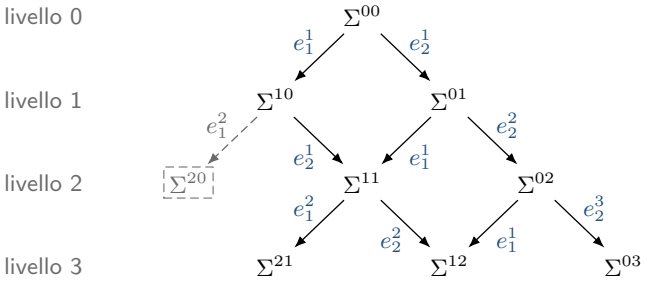


Figura 2.4. I primi livelli del reticolo della Figura 2.3, con l'evento eseguito a ogni passo. Da Σ^{10} non si può eseguire e_1^2, che riceve un messaggio non ancora inviato: Σ^{20} non è consistente.

- per il messaggio da e_2^1 a e_1^2: se $a \geq 2$, allora $b \geq 1$;
- per il messaggio da e_2^3 a e_1^5: se $a \geq 5$, allora $b \geq 3$;
- per il messaggio da e_1^3 a e_2^5: se $b \geq 5$, allora $a \geq 3$.

Nel reticolo completo della Figura 2.3 mancano quindi gli stati con $a \geq 2$ e $b = 0$, come Σ^{20}; quelli con $a \geq 5$ e $b \leq 2$, come Σ^{51}; e quelli con $b = 5$ e $a \leq 2$, come Σ^{05}. Tutti gli altri ci sono.

Cammini e run. Un **cammino** nel reticolo è una sequenza di stati in cui ogni stato ha un evento in più del precedente: a ogni passo si sceglie quale processo avanza. Ogni cammino dallo stato iniziale a quello finale corrisponde a un run consistente, e ogni run consistente è un cammino. Il cammino evidenziato in blu nella Figura 2.3, per esempio, passa per gli stati

$$\Sigma^{00}\,\Sigma^{01}\,\Sigma^{11}\,\Sigma^{21}\,\Sigma^{31}\,\Sigma^{32}\,\Sigma^{42}\,\Sigma^{43}\,\Sigma^{44}\,\Sigma^{54}\,\Sigma^{64}\,\Sigma^{65}$$

e corrisponde al run

$$e_2^1\,e_1^1\,e_1^2\,e_1^3\,e_2^2\,e_1^4\,e_2^3\,e_2^4\,e_1^5\,e_1^6\,e_2^5.$$

Ogni passo a sinistra è un evento di p_1, ogni passo a destra un evento di p_2. Cammini diversi sono ordini diversi in cui si possono eseguire gli eventi concorrenti, e descrivono tutti la stessa computazione. L'esecuzione reale è uno di questi cammini, ma dall'interno del sistema non si può sapere quale (Sezione 2.3).

Raggiungibilità. Nel reticolo, $\Sigma \rightsquigarrow \Sigma'$ vuol dire che da Σ si può scendere fino a Σ' seguendo gli archi. Succede esattamente quando in Σ' ogni processo ha eseguito almeno tanti eventi quanti in Σ. Per esempio $\Sigma^{21} \rightsquigarrow \Sigma^{23}$, passando per Σ^{22}; invece tra Σ^{31} e Σ^{23} nessuno dei due è raggiungibile dall'altro, perché il primo ha più eventi di p_1 e il secondo più eventi di p_2.

Tre proprietà dei tagli consistenti aiutano a ragionare sul reticolo.

- *Intersezione e unione di due tagli consistenti sono tagli consistenti.* Se un evento sta in entrambi i tagli, le sue cause stanno in entrambi; se sta in almeno uno dei due, le sue cause stanno almeno in quello. È questo a fare dell'insieme un reticolo: per due stati consistenti esistono sempre il più grande stato da cui entrambi sono raggiungibili (quello dell'intersezione, il minimo componente per componente) e il più piccolo stato raggiungibile da entrambi (quello dell'unione, il massimo). Per Σ^{31} e Σ^{23} sono Σ^{21} e Σ^{33}.
- *Ogni stato globale consistente è raggiungibile dallo stato iniziale con un run consistente.* Basta eseguire gli eventi del taglio uno alla volta, scegliendo ogni volta un evento le cui cause sono già state eseguite. Un evento così esiste sempre, perché la precedenza causale non ha cicli.
- *Se $C \subseteq C'$ sono due tagli consistenti, lo stato di C' è raggiungibile da quello di C.* Si prende un run consistente che arriva allo stato di C e lo si prolunga con gli eventi di $C' \setminus C$, sempre in un ordine che rispetta la precedenza causale.

L'ultima proprietà tornerà utile con gli snapshot (Sezione 5.6).

Capitolo 3

Osservare una computazione: orologio reale e clock logici

Nel capitolo precedente il monitor interrogava attivamente i processi e otteneva un taglio qualunque. Qui si prova la strategia opposta: il monitor non chiede nulla, sono i processi a informarlo di ogni evento, e il suo compito diventa mettere in ordine le notifiche in modo che la sequenza osservata sia consistente. Si parte da ipotesi forti, un orologio globale e ritardi limitati, e le si toglie una alla volta. Lo strumento centrale del capitolo sono i clock logici di Lamport.

3.1 Monitoraggio passivo e osservazioni

Nella strategia passiva il monitor p_0 non invia messaggi. I processi della computazione vengono modificati di poco: ogni volta che eseguono un evento, lo notificano a p_0 con un messaggio che lo descrive. L'invio della notifica non è un evento nuovo, ma coincide con l'evento notificato: così, come nella strategia attiva (Sezione 2.2), il monitoraggio non aggiunge eventi alla computazione e non ne cambia la numerazione. In pratica basta notificare gli eventi **rilevanti** per Φ, cioè quelli che possono cambiarne il valore: per il deadlock della computazione d'esempio sono gli invii e le ricezioni di richieste e risposte, gli unici che modificano il grafo delle attese. Il monitor costruisce così un'**osservazione** della computazione: la sequenza degli eventi nell'ordine in cui arrivano le notifiche.

Le osservazioni hanno due caratteristiche importanti.

- Poiché i ritardi variano, la stessa esecuzione può produrre osservazioni diverse su monitor diversi. È il cosiddetto **effetto relativistico** del calcolo distribuito: non esiste un punto di vista privilegiato.
- Un'osservazione può corrispondere a un run consistente, a un run inconsistente o a nessun run, se le notifiche di uno stesso processo arrivano in disordine. Si dice **consistente** un'osservazione che corrisponde a un run consistente.

Esempio (tre osservazioni della stessa esecuzione). Supponiamo che la computazione d'esempio (Figura 1.2) venga notificata a p_0. Con canali non FIFO

qualunque ordine di arrivo delle notifiche è possibile.

- Se la notifica di e_3^4 arriva prima di quella di e_3^3, la sequenza osservata non rispetta nemmeno la storia locale di p_3: non è un run.
- Se le notifiche di e_1^4 ed e_1^5 tardano mentre quella di e_3^6 arriva presto, p_0 vede p_3 ricevere una richiesta che p_1 non ha ancora inviato: l'osservazione è un run inconsistente. Se a quel punto p_0 valuta Φ, si trova proprio nello stato del taglio C', cioè $(\sigma_1^3, \sigma_2^2, \sigma_3^6)$, e ritrova il deadlock fantasma.
- Se le notifiche arrivano in un ordine che rispetta la precedenza causale, l'osservazione è consistente, e tutti gli stati che p_0 attraversa sono consistenti: per esempio quello del taglio C.

3.2 Regole di consegna e consegna FIFO

Per rimettere in ordine i messaggi, chi li riceve non li passa subito all'applicazione, cioè al proprio programma, ma decide quando farlo seguendo una **regola di consegna** (*delivery rule*). Ogni processo che riceve messaggi ha quindi due livelli, come la portineria e la scrivania di un ufficio: il *livello di consegna*, che riceve i messaggi dal canale e li tiene in attesa, e l'*applicazione*, che li usa. Le due operazioni sono *receive*, con cui il messaggio arriva dal canale, e *deliver*, con cui viene consegnato all'applicazione: avvengono entrambe nello stesso processo, e consegnare non vuol dire spedire a qualcun altro. La ricezione resta nascosta dentro la regola e non compare nella storia locale: per l'applicazione conta solo la consegna. Per il monitor (Figura 3.1) le notifiche arrivano al livello di consegna di p_0, che le passa al programma di p_0 nell'ordine stabilito dalla regola. D'ora in poi, quindi, l'osservazione è la sequenza delle notifiche *consegnate*, non di quelle ricevute.

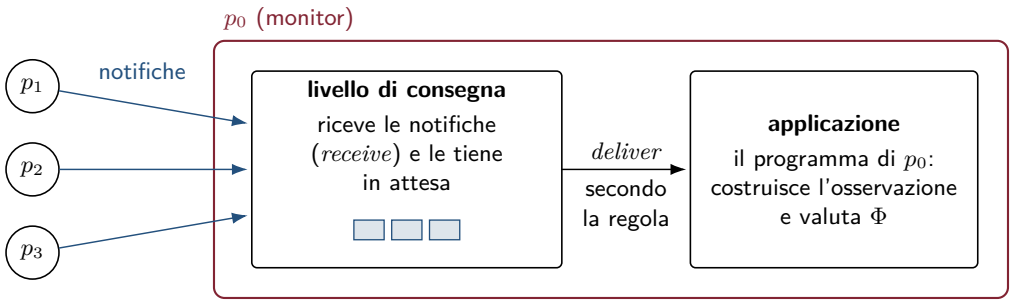


Figura 3.1. Il monitor nella strategia passiva. Le notifiche (in blu) arrivano al livello di consegna di p_0 e aspettano; la regola di consegna le passa, nell'ordine giusto, all'applicazione di p_0, che costruisce l'osservazione e valuta Φ. I due livelli fanno parte dello stesso processo.

Definizione (consegna FIFO). La comunicazione da p_i a p_j rispetta la **consegna FIFO** (*First In, First Out*) se, per ogni coppia di messaggi m ed m',

$$send_i(m) \to send_i(m') \;\Rightarrow\; deliver_j(m) \to deliver_j(m'),$$

dove il pedice indica il processo che esegue l'evento.

In parole: un messaggio non può sorpassare un messaggio inviato prima dallo stesso mittente. La consegna FIFO si realizza facilmente anche su canali che non la garantiscono: il mittente numera i messaggi che invia a ciascun destinatario, e il destinatario li consegna in ordine di numero, tenendo da parte quelli arrivati in anticipo. Se per esempio arrivano prima i messaggi numero 2 e 3, il destinatario li trattiene finché non arriva l'1, e poi consegna 1, 2 e 3.

Con canali FIFO tra i processi e il monitor, le notifiche di ogni processo arrivano nell'ordine giusto e ogni osservazione è un run: il primo caso dell'esempio non può più capitare. La consistenza però non è garantita: il secondo caso riguarda notifiche di processi *diversi*, p_1 e p_3, che la consegna FIFO non ordina tra loro. Serve un meccanismo che ordini le notifiche secondo la precedenza causale.

3.3 Orologio reale e clock condition

Supponiamo che tutti i processi possano leggere un orologio reale globale e che ogni messaggio arrivi entro un ritardo massimo noto δ. Non è più un sistema asincrono, ma è un buon punto di partenza. Sia $RC(e)$ il valore dell'orologio quando viene eseguito l'evento e: ogni notifica porta con sé come **timestamp** (marca temporale) il valore $RC(e)$ dell'evento notificato, e il monitor segue questa regola.

Definizione (regola DR1). Al tempo t, consegna tutti i messaggi ricevuti con timestamp fino a $t - \delta$, in ordine di timestamp crescente.

Perché funziona? Un messaggio con timestamp non superiore a $t - \delta$ è stato inviato almeno δ prima di t, quindi al tempo t è sicuramente già arrivato. Per esempio, con $\delta = 3$, al tempo 10 il monitor consegna i messaggi con timestamp fino a 7: uno con timestamp 7 è partito al tempo 7 ed è arrivato entro il 10, quindi nessun messaggio con timestamp fino a 7 può ancora arrivare. Quelli con timestamp 8, 9 e 10 aspettano: un messaggio con timestamp 8 può arrivare fino al tempo 11, e se il monitor avesse già consegnato un 9 l'ordine sarebbe sbagliato. L'osservazione ordina quindi gli eventi per timestamp crescente: e viene osservato prima di e' se e solo se $RC(e) < RC(e')$ (a parità di timestamp decide l'indice del processo). Allora l'osservazione è consistente se e solo se i timestamp rispettano la precedenza causale.

Definizione (clock condition). Una funzione T che associa un valore a ogni evento soddisfa la **clock condition** se

$$e \to e' \;\Rightarrow\; T(e) < T(e').$$

L'orologio reale la soddisfa senza fatica, perché una causa avviene sempre prima del suo effetto. L'osservazione chiave è che la clock condition è l'*unica* proprietà dell'orologio reale che serve: qualunque meccanismo di timestamp che la soddisfi si può usare al posto del tempo reale. E un meccanismo così esiste anche in un sistema asincrono.

3.4 Clock logici di Lamport

Il clock logico di Lamport è un contatore che ogni processo mantiene per conto proprio e che fa da orologio senza alcuna sincronizzazione tra i processi.

Definizione (clock logico di Lamport). Ogni processo p_i mantiene un contatore LC, il suo **clock logico** (*logical clock*), inizialmente 0. Ogni messaggio m porta un timestamp $TS(m)$, il valore del clock del mittente nell'evento di invio. A ogni nuovo evento e_i, p_i aggiorna il clock così:

$$LC(e_i) = \begin{cases} LC + 1 & \text{se } e_i \text{ è un evento interno o un invio,} \\ \max\{LC, TS(m)\} + 1 & \text{se } e_i = \mathit{receive}(m). \end{cases}$$

Alla ricezione il clock salta oltre il timestamp del messaggio: così il destinatario non resta indietro rispetto al passato del mittente.

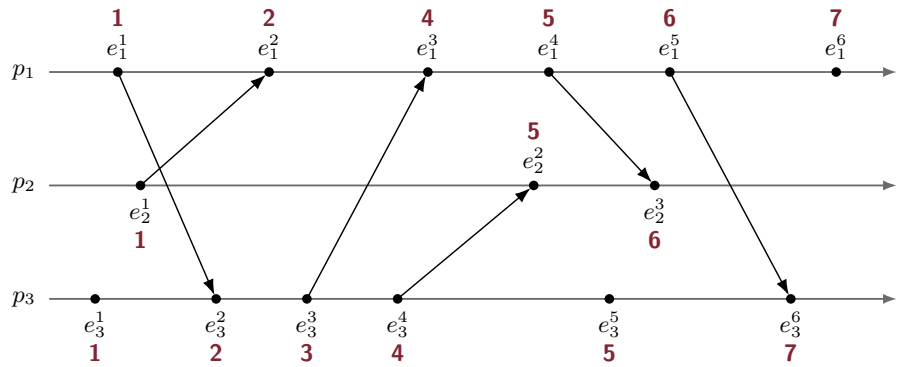


Figura 3.2. I clock logici della computazione d'esempio: accanto a ogni evento, in rosso, il valore di LC.

Esempio (clock logici passo passo). Applichiamo le regole alla computazione d'esempio (Figura 3.2). All'inizio tutti i clock valgono 0.

- I primi eventi e_1^1, e_2^1 ed e_3^1 sono invii o eventi interni, e il loro clock vale 1. Le richieste inviate in e_1^1 ed e_2^1 portano timestamp 1.
- e_1^2 riceve la richiesta di p_2, con timestamp 1: $\max\{1, 1\} + 1 = 2$. Allo stesso modo e_3^2, che riceve la richiesta di p_1, vale 2.
- p_3 invia la risposta, $LC(e_3^3) = 3$, e poi una richiesta a p_2, $LC(e_3^4) = 4$.
- e_1^3 riceve la risposta di p_3, con timestamp 3: $\max\{2, 3\} + 1 = 4$. Il clock di p_1 salta da 2 a 4 per superare il passato di p_3.
- e_2^2 riceve la richiesta di p_3, con timestamp 4: $\max\{1, 4\} + 1 = 5$.
- p_1 prosegue con la risposta a p_2, $LC(e_1^4) = 5$, la richiesta a p_3, $LC(e_1^5) = 6$, e l'evento interno e_1^6, che vale 7.

- e_2^3 riceve la risposta di p_1, con timestamp 5: $\max\{5,5\}+1=6$.
- e_3^5 è interno e vale 5; e_3^6 riceve la richiesta inviata in e_1^5, con timestamp 6: $\max\{5,6\}+1=7$. Il clock di p_3 passa da 5 a 7 e non assume mai il valore 6.

I clock logici crescono lungo ogni catena causale: a ogni passo dentro un processo il clock aumenta di uno, e a ogni messaggio il clock del destinatario supera il timestamp del mittente. Ne segue la proprietà fondamentale.

I clock logici soddisfano la clock condition. Per ogni coppia di eventi

$$e \to e' \;\Rightarrow\; LC(e) < LC(e').$$

La clock condition vale in un verso solo: un clock più piccolo non indica per forza una causa, e dai soli valori dei clock non si capisce se due eventi siano legati o concorrenti. Nella Figura 3.2, $LC(e_2^2)=5<LC(e_3^6)=7$, ma i due eventi sono concorrenti, come si è visto nel capitolo 1. Da $LC(e)<LC(e')$ si deduce soltanto che e' non precede causalmente e.

Ordinando tutti gli eventi per clock logico crescente, e risolvendo le parità con l'indice del processo, si ottiene un ordine totale che rispetta la precedenza causale, cioè un run consistente. Per la computazione d'esempio è

$$e_1^1\, e_2^1\, e_3^1\, e_1^2\, e_3^2\, e_3^3\, e_1^3\, e_3^4\, e_1^4\, e_2^2\, e_3^5\, e_1^5\, e_2^3\, e_1^6\, e_3^6.$$

3.5 Consegnare le notifiche in ordine di clock logico

Torniamo al monitor. Se i processi usano i clock logici come timestamp delle notifiche e p_0 le consegna in ordine di timestamp crescente, l'osservazione è consistente, perché i clock logici soddisfano la clock condition. Resta però un problema serio: senza un limite ai ritardi, p_0 non può mai essere sicuro che più tardi non arrivi una notifica con timestamp minore di quelle che ha in mano. Per prudenza non consegnerebbe mai nulla: la regola non ha *liveness*, cioè non garantisce che prima o poi qualcosa venga consegnato. Manca la capacità di accorgersi dei «buchi».

Definizione (gap detection). Un meccanismo di clock ha la proprietà di **gap detection** se, dati due eventi e ed e' con $LC(e)<LC(e')$, permette di stabilire se esiste un altro evento e'' con $LC(e)<LC(e'')<LC(e')$.

I clock logici non hanno questa proprietà: i valori possono saltare, come il clock di p_1 che passa da 2 a 4, e guardando i soli timestamp non si sa se da qualche parte ci sia un evento con valore 3 di cui non è ancora arrivata la notifica. Nemmeno l'orologio reale ha la gap detection, ma con il limite δ sui ritardi il controllo diventa facile: al tempo t sono arrivati tutti i messaggi con timestamp minore di $t-\delta$.

Con i clock logici serve un'informazione in più, e la fornisce la consegna FIFO. Supponiamo che i canali da ogni processo a p_0 siano FIFO. Poiché il clock di ogni processo cresce e i messaggi di uno stesso mittente arrivano in ordine, quando p_0

riceve da p_i un messaggio con un certo timestamp sa che da p_i non arriveranno più messaggi con timestamp minore o uguale. Un messaggio m ricevuto da p_0 si dice **stabile** se non può più arrivare un messaggio con timestamp minore di $TS(m)$. Con canali FIFO, m è stabile non appena p_0 ha ricevuto da *ogni* altro processo almeno un messaggio con timestamp maggiore di $TS(m)$. Per esempio, p_0 ha una notifica di p_1 con timestamp 4, e ha già ricevuto un 6 da p_2 e un 5 da p_3: né p_2 né p_3 manderanno più niente sotto il 4, quindi la notifica è stabile. Se da p_3 avesse ricevuto solo un 2, da p_3 potrebbe ancora arrivare un 3, e p_0 dovrebbe aspettare.

Definizione (regola DR2). Consegna tutti i messaggi ricevuti che sono stabili per p_0, in ordine di timestamp crescente.

I due ingredienti hanno ruoli diversi: i clock logici danno l'ordine, perché una causa ha sempre un timestamp più piccolo dei suoi effetti, e la consegna FIFO dà la certezza che non arriverà più niente di più piccolo. La regola si può ancora bloccare se un processo smette di inviare notifiche, perché p_0 non riceverebbe mai da lui un timestamp più grande. Si rimedia facendo chiedere a p_0, periodicamente, una conferma a tutti i processi: le risposte hanno timestamp più grandi e rendono stabili i messaggi precedenti.

DR2 funziona, ma a volte aspetta senza bisogno (Sezione 4.2); il capitolo 4 mostra come evitarlo.

Capitolo 4

Consegna causale e vector clock

Nel capitolo precedente il monitor p_0 metteva in ordine le notifiche dei processi con i clock logici, così da osservare un run consistente, ma a volte aspettava senza bisogno. Questo capitolo introduce la consegna causale, cioè l'ordine di cui il monitor ha davvero bisogno, e i *vector clock* (clock vettoriali): timestamp da cui si capisce sempre se due eventi sono legati causalmente o concorrenti, e con cui la consegna causale si realizza senza attese inutili.

4.1 Consegna causale

Un messaggio viene *ricevuto* quando arriva dal canale, e *consegnato* quando il processo lo passa all'applicazione (Sezione 3.2). La consegna FIFO mette in ordine i messaggi di uno stesso mittente; con mittenti diversi non basta.

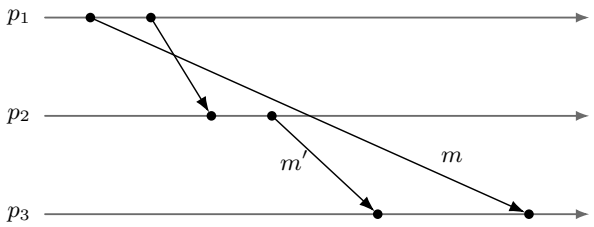


Figura 4.1. Consegna FIFO ma non causale: l'invio di m precede causalmente quello di m', ma p_3 riceve m' per primo.

Nella Figura 4.1, p_1 invia m a p_3 e poi un messaggio a p_2, che dopo averlo ricevuto invia m' a p_3. L'invio di m precede causalmente quello di m': c'è un cammino che parte dall'invio di m, segue p_1 fino all'invio a p_2, scende lungo la freccia e prosegue su p_2 fino all'invio di m'. Quindi m' *potrebbe* dipendere da m. Se invece p_1 scrivesse a p_2 prima di inviare m, il cammino non ci sarebbe, e i due invii sarebbero concorrenti.

Eppure m' arriva a p_3 per primo. La consegna FIFO è rispettata, perché su ogni canale viaggia un solo messaggio; ma se p_3 consegna subito m', vede l'effetto prima della causa, come chi in una chat di gruppo leggesse la risposta prima della domanda. La consegna causale lo vieta: se l'invio di un messaggio precede causalmente l'invio di un altro, e i due vanno allo stesso destinatario, il primo va consegnato prima.

Definizione (consegna causale). Una regola di consegna rispetta la **consegna causale** (*causal delivery*, CD) se, per ogni coppia di messaggi m ed m' diretti allo stesso processo p_k,

$$send_i(m) \rightarrow send_j(m') \;\Rightarrow\; deliver_k(m) \rightarrow deliver_k(m'),$$

dove il pedice indica il processo che esegue l'evento: p_i invia m, p_j invia m' e p_k consegna entrambi.

Con lo stesso mittente ($i = j$) si ritrova la consegna FIFO, perché due invii dello stesso processo sono sempre legati causalmente. La consegna causale è quindi una FIFO più forte: ordina anche i messaggi di mittenti diversi, ma solo se i loro invii sono legati causalmente; quelli con invii concorrenti si possono consegnare in qualunque ordine.

È esattamente ciò che serve al monitor. La notifica di un evento parte in quell'evento, quindi se $e \rightarrow e'$ l'invio della notifica di e precede causalmente quello della notifica di e': con la consegna causale il monitor vede sempre le cause prima degli effetti, e tutte le sue osservazioni sono consistenti.

La consegna causale dice *che cosa* garantire, non *come*. DR1 e DR2 del capitolo precedente la realizzano già, con un orologio globale e ritardi limitati oppure con canali FIFO e clock logici, ma sono troppo prudenti: ordinano per timestamp anche le notifiche di eventi concorrenti, e DR2 aspetta un timestamp più grande da tutti i processi. Il resto del capitolo costruisce una realizzazione che impone solo l'ordine richiesto, in un sistema asincrono: serve un timestamp che distingua le cause dagli eventi concorrenti (Sezione 4.2), che si ottiene dalle storie causali (Sezione 4.3) riassunte nei vector clock (Sezioni 4.4 e 4.5), usati dalla regola DR3 (Sezione 4.6).

4.2 La strong clock condition

DR1 e DR2 consegnano in ordine di timestamp crescente, quindi trattano ogni notifica con un timestamp più piccolo come una possibile causa. Con i clock logici della Figura 3.2, $LC(e_2^2) = 5$ e $LC(e_3^6) = 7$: se a p_0 arriva prima la notifica di e_3^6, deve aspettare quella di e_2^2, che potrebbe essere una sua causa. In realtà i due eventi sono concorrenti (capitolo 1), e l'attesa è inutile. Il valore 5, da solo, non dice se e_2^2 è una causa di e_3^6 o un evento concorrente: serve un timestamp da cui si capisca *esattamente* quando due eventi sono legati.

Definizione (strong clock condition). Una funzione T che associa un timestamp a ogni evento soddisfa la **strong clock condition** se, per ogni coppia di eventi,

$$e \rightarrow e' \iff T(e) < T(e').$$

Il verso da sinistra a destra è la clock condition del capitolo 3; quello nuovo dice che un timestamp più piccolo indica davvero una causa. Così il confronto tra due timestamp dice sempre come stanno due eventi: se uno dei due è più piccolo, quell'evento precede l'altro; se nessuno dei due lo è, sono concorrenti. Si dice che un

timestamp così **caratterizza** la precedenza causale: dai soli timestamp si ricostruisce tutta la relazione $\rightarrow$.

Un solo numero non basta: due numeri diversi sono sempre confrontabili, mentre i timestamp di due eventi concorrenti non devono esserlo. Servono timestamp con più dimensioni, che possano risultare incomparabili: è l'idea dei vector clock, a cui si arriva passando per le storie causali.

4.3 Storie causali

Il modo più diretto per soddisfare la strong clock condition è usare come timestamp di un evento tutto il suo passato, cioè l'insieme degli eventi che possono averlo influenzato.

Definizione (storia causale). La **storia causale** (*causal history*) di un evento e è l'insieme formato da e e da tutti gli eventi che lo precedono causalmente:

$$\theta(e) = \{\, e' \in H \mid e' \rightarrow e \,\} \cup \{e\},$$

dove θ è la lettera greca *theta* e H è la storia globale, cioè l'insieme di tutti gli eventi.

Nella Figura 4.2 è evidenziata la storia causale di e_1^4:

$$\theta(e_1^4) = \{e_1^1, e_1^2, e_1^3, e_1^4, e_2^1, e_3^1, e_3^2, e_3^3\}.$$

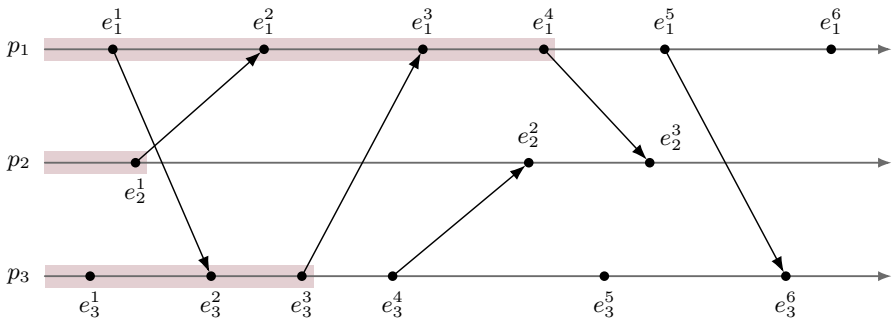


Figura 4.2. La storia causale di e_1^4: i tratti evidenziati delle storie locali.

Le storie causali si confrontano per contenimento: $e \rightarrow e'$ se e solo se $\theta(e) \subset \theta(e')$, e se nessuna delle due contiene l'altra gli eventi sono concorrenti. La strong clock condition è soddisfatta. Il difetto è pratico: le storie causali crescono senza limite, e i messaggi dovrebbero trasportare insiemi sempre più grandi.

4.4 Vector clock

Una storia causale si può riassumere con pochi numeri. Su ogni processo contiene sempre i primi eventi, senza buchi: se contiene un evento di p_3, contiene anche quelli precedenti di p_3, perché precedono causalmente anche loro. Di p_3, per esempio, la

storia causale di e_1^4 contiene e_3^1, e_3^2 ed e_3^3. Basta quindi dire, per ogni processo, quanti eventi contiene: per e_1^4 sono 4 di p_1, 1 di p_2 e 3 di p_3, cioè il vettore $(4, 1, 3)$.

Definizione (vector clock). Il **vector clock** di un evento e è il vettore $VC(e)$ di n numeri naturali, uno per processo: la componente i, indicata con $VC(e)[i]$, è il numero di eventi di p_i che stanno nella storia causale di e.
Si calcola mentre la computazione procede. Ogni processo p_i tiene un vettore VC, all'inizio tutto a zero, e ogni messaggio m porta come timestamp $TS(m)$ il vector clock dell'evento in cui è stato inviato. A ogni evento, p_i aggiorna il vettore così:

- se l'evento è interno o un invio, incrementa la propria componente: $VC[i] := VC[i] + 1$;
- se l'evento è la ricezione di un messaggio m, prima prende il massimo componente per componente tra il proprio vettore e il timestamp, $VC := \max\{VC, TS(m)\}$, e poi incrementa la propria componente.

Qui VC senza argomento è il vettore corrente del processo, che dopo un evento e vale $VC(e)$; := è un assegnamento, e il massimo si prende posizione per posizione, per esempio $\max\{(2, 1, 0), (1, 0, 3)\} = (2, 1, 3)$: chi riceve unisce ciò che sapeva con ciò che sapeva il mittente. Per un evento e di p_i, quindi, $VC(e)[i]$ è la posizione di e tra gli eventi di p_i, e ogni altra componente $VC(e)[j]$ dice quanti eventi di p_j precedono causalmente e, cioè quanto p_i «sa» di p_j in quel momento.

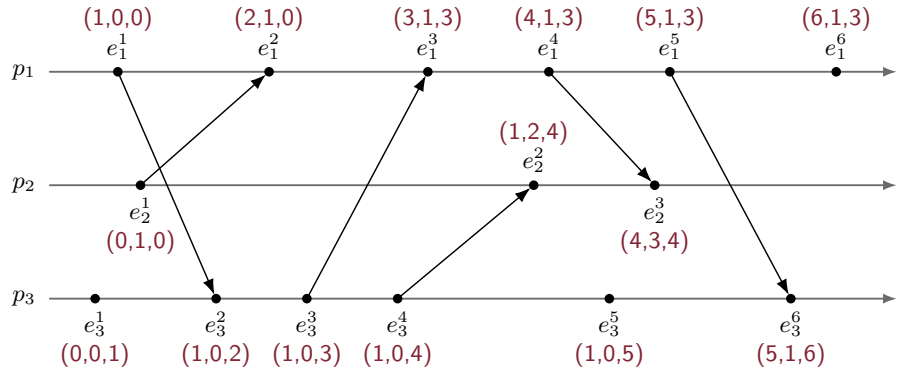


Figura 4.3. I vector clock della computazione d'esempio: accanto a ogni evento, in rosso, il suo vector clock.

Esempio (vector clock passo passo). Applichiamo le regole alla computazione d'esempio (Figura 4.3); i vettori hanno tre componenti, una per processo.

- e_1^1, e_2^1 ed e_3^1 sono i primi eventi dei rispettivi processi, con vector clock $(1, 0, 0)$, $(0, 1, 0)$ e $(0, 0, 1)$.
- e_1^2 riceve la richiesta di p_2, con timestamp $(0, 1, 0)$: il massimo con $(1, 0, 0)$ è $(1, 1, 0)$, e incrementando la componente di p_1 si ottiene $(2, 1, 0)$.

- e_3^2 riceve la richiesta di p_1, con timestamp $(1,0,0)$: dal massimo con $(0,0,1)$, cioè $(1,0,1)$, si ottiene $(1,0,2)$. Seguono due invii, con vector clock $(1,0,3)$ e $(1,0,4)$.
- e_1^3 riceve la risposta di p_3, con timestamp $(1,0,3)$: dal massimo con $(2,1,0)$, cioè $(2,1,3)$, si ottiene $(3,1,3)$. Gli eventi successivi di p_1 non ricevono nulla e incrementano solo la prima componente: $(4,1,3)$, $(5,1,3)$, $(6,1,3)$.
- e_2^2 riceve la richiesta di p_3, con timestamp $(1,0,4)$, e vale $(1,2,4)$; e_2^3 riceve la risposta di p_1, con timestamp $(4,1,3)$: il massimo con $(1,2,4)$ è $(4,2,4)$, quindi $(4,3,4)$.
- e_3^5 è interno, $(1,0,5)$; e_3^6 riceve la richiesta di p_1, con timestamp $(5,1,3)$, e vale $(5,1,6)$.

Per esempio $VC(e_2^3) = (4,3,4)$ dice che e_2^3 è il terzo evento di p_2 e che lo precedono causalmente quattro eventi di p_1 (e_1^1–e_1^4) e quattro di p_3 (e_3^1–e_3^4).

4.5 Confrontare i vector clock

Due vettori V e V' di n numeri si confrontano posizione per posizione: $V \leq V'$ se ogni componente di V è minore o uguale a quella di V', e $V < V'$ se in più i due vettori sono diversi. Per esempio $(1,0,0) < (2,1,0)$. A differenza dei numeri, due vettori possono essere **incomparabili**: tra $(2,1,0)$ e $(1,0,4)$ nessuno è minore dell'altro, perché il primo è più grande nella prima componente e il secondo nella terza. È proprio ciò che serve per rappresentare la concorrenza. Ecco che cosa si capisce dai vector clock, con i valori della Figura 4.3.

Un evento precede l'altro? Sì, se e solo se il suo vector clock è più piccolo: i vector clock soddisfano la strong clock condition,

$$e \to e' \iff VC(e) < VC(e').$$

Infatti e precede e' se e solo se il passato di e sta tutto nel passato di e', cioè se su ogni processo e' conosce almeno tanti eventi quanti ne conosce e. Per esempio $VC(e_2^1) = (0,1,0) < (5,1,6) = VC(e_3^6)$, quindi $e_2^1 \to e_3^6$.

Due eventi sono concorrenti? Sì, se e solo se i loro vector clock sono incomparabili, come $VC(e_2^2) = (1,2,4)$ e $VC(e_3^6) = (5,1,6)$.

Se si sa di quali processi sono i due eventi, basta una componente. Se e è il k-esimo evento di p_i, cioè $VC(e)[i] = k$, ed e' è un evento di un altro processo, e precede e' se e solo se e' conosce almeno i primi k eventi di p_i:

$$e \to e' \iff VC(e')[i] \geq VC(e)[i].$$

Per esempio e_1^4 è il quarto evento di p_1 e $VC(e_2^3)[1] = 4$, quindi $e_1^4 \to e_2^3$; invece e_2^2 è il secondo evento di p_2, ma e_3^6 ne conosce uno solo, quindi $e_2^2 \not\to e_3^6$.

Un taglio è consistente? Basta guardare la frontiera, cioè l'ultimo evento di ogni processo nel taglio. Sia $(c_1, \ldots, c_n)$ il taglio, dove c_i è il numero di eventi di p_i che contiene. Il taglio è consistente se e solo se nessun evento della frontiera conosce, di un processo, più eventi di quanti il taglio ne contenga, cioè se il vector clock di ogni evento della frontiera è minore o uguale alla tupla del taglio:

$$VC(e_j^{c_j}) \leq (c_1, \ldots, c_n) \qquad \text{per ogni processo } p_j,$$

dove $e_j^{c_j}$ è l'ultimo evento di p_j nel taglio (si considerano solo i processi con almeno un evento nel taglio). Se un evento della frontiera conoscesse più eventi di un processo di quanti il taglio ne contiene, dipenderebbe da un evento rimasto fuori. Per i due tagli della Figura 2.1:

- per $C = (5, 2, 4)$ i vettori della frontiera sono $(5, 1, 3)$, $(1, 2, 4)$ e $(1, 0, 4)$, tutti minori o uguali a $(5, 2, 4)$: il taglio è consistente;
- per $C' = (3, 2, 6)$ il vettore di e_3^6, $(5, 1, 6)$, supera il 3 nella prima componente: e_3^6 conosce cinque eventi di p_1, ma il taglio ne contiene tre, quindi C' è inconsistente.

Eventi come e_1^3 ed e_3^6, che non possono stare insieme sulla frontiera di un taglio consistente, si dicono **inconsistenti a coppie** (*pairwise inconsistent*).

Manca qualcosa in mezzo? Prima di consegnare una notifica, il monitor deve sapere se gli manca ancora qualcosa da consegnare prima. Con i clock logici questa domanda, la gap detection (Sezione 3.5), non ha risposta, perché i valori saltano: chi ha consegnato fino al 5 e riceve un 7 non sa se esiste un 6 ancora in viaggio. I vector clock rispondono a una domanda più modesta, che però basta: la **weak gap detection**.

$VC(e)[k] = c$ vuol dire che e conosce i primi c eventi di p_k. Se e ne conosce meno di e', gli eventi di p_k in più stanno nel passato di e' ma non in quello di e:

$$VC(e)[k] < VC(e')[k] \implies \text{esiste un evento } f \text{ di } p_k \text{ con } f \not\to e \text{ e } f \to e'.$$

La proprietà è debole perché «e non conosce f» non vuol dire «f viene dopo e». Con $VC(e_1^2) = (2, 1, 0)$ e $VC(e_2^3) = (4, 3, 4)$:

- nella terza componente $0 < 4$, ma un evento trovato così, come e_3^1, è concorrente a e_1^2: non sta *tra* i due;
- nella prima componente $2 < 4$, e gli eventi trovati, e_1^3 ed e_1^4, sono dello stesso processo di e_1^2, quindi vengono dopo: stanno proprio tra i due.

Il monitor della Sezione 4.6 è sempre nel secondo caso, perché per ogni processo p_k confronta la notifica arrivata con l'ultima notifica di p_k già consegnata, cioè con un evento di p_k stesso. Se la notifica arrivata conosce le prime 3 notifiche di p_1 e il monitor ne ha consegnate 2, manca la terza, che va consegnata prima: il monitor sa *che* manca qualcosa, *di chi* è e *quanto* manca.

4.6 Consegna causale con i vector clock

Torniamo al monitor. Ogni notifica porta come timestamp il vector clock dell'evento notificato, e i processi incrementano la propria componente solo negli eventi che

notificano (o, che è lo stesso, notificano tutti i loro eventi): così le componenti contano notifiche, e il monitor, che vede solo quelle, le può confrontare con quelle che ha ricevuto. Per una notifica m inviata da p_j:

- $TS(m)[j]$ è la posizione di m tra le notifiche di p_j (m è la prima, la seconda, ...);
- $TS(m)[k]$, per ogni altro processo p_k, è il numero di notifiche di p_k che precedono causalmente m, cioè quante ne conosceva p_j quando ha inviato m.

Il monitor tiene da parte le notifiche ricevute ma non ancora consegnate, e un contatore per ogni processo: $D[k]$, inizialmente 0, è il numero di notifiche di p_k già consegnate.

Il monitor può consegnare una notifica m di p_j quando nessun'altra notifica, arrivata o ancora in viaggio, deve venire prima. Per esserne sicuro risponde a due domande.

1. *È la prossima notifica di p_j?* Le notifiche di p_j vanno consegnate nell'ordine di invio: se m è la terza, il monitor deve averne consegnate esattamente due.
2. *Ha già consegnato tutto ciò che p_j conosceva?* Le notifiche degli altri processi che p_j conosceva quando ha inviato m precedono causalmente m, quindi vanno consegnate prima. Se ne manca qualcuna, la weak gap detection lo rivela, con il buco su p_k.

Se una risposta è no, la notifica aspetta, e il monitor la riesamina ogni volta che consegna qualcos'altro.

Definizione (regola DR3, consegna causale). Il monitor consegna la notifica m di p_j non appena

$$D[j] = TS(m)[j] - 1 \qquad \text{e} \qquad D[k] \geq TS(m)[k] \ \text{ per ogni } k \neq j,$$

e dopo la consegna pone $D[j] := TS(m)[j]$.

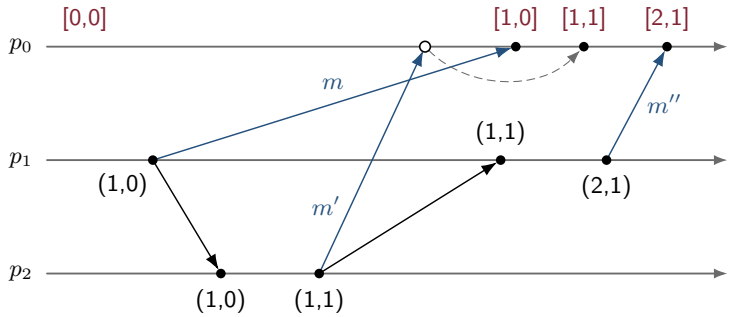


Figura 4.4. La regola DR3 al lavoro. Su p_1 e p_2 i vector clock, su p_0 (in rosso) i valori di D; in blu le notifiche. m' arriva prima di m (cerchio vuoto) e viene consegnata solo dopo di essa.

Esempio (la regola DR3 al lavoro). Nella Figura 4.4 ci sono due processi, p_1 e p_2, e il monitor p_0. I vector clock hanno due componenti, una per processo, e si incrementano solo negli eventi notificati.

- p_1 esegue un evento notificato, con vector clock $(1,0)$, in cui invia un messaggio a p_2; la sua notifica è m, la prima di p_1.
- p_2 riceve il messaggio. L'evento non è notificato, quindi il vector clock diventa $\max\{(0,0),(1,0)\} = (1,0)$ senza incremento. Poi p_2 esegue un evento notificato, con vector clock $(1,1)$, in cui invia un messaggio a p_1; la sua notifica è m', la prima di p_2.
- m' arriva a p_0 per prima, con $TS(m') = (1,1)$ e $D = [0,0]$. È la prossima notifica di p_2 ($D[2] = 0 = 1-1$), ma p_2 conosceva una notifica di p_1 che il monitor non ha ancora consegnato ($D[1] = 0 < 1$): m' resta in attesa.
- Arriva m, con $TS(m) = (1,0)$: è la prossima notifica di p_1 ($D[1] = 0 = 1-1$) e p_1 non conosceva notifiche di p_2 ($D[2] = 0 \geq 0$), quindi m viene consegnata e D diventa $[1,0]$. Ora anche m' supera la seconda domanda ($D[1] = 1 \geq 1$): viene consegnata e D diventa $[1,1]$.
- p_1 riceve il messaggio di p_2 (vector clock $(1,1)$) e poi esegue un evento notificato, $(2,1)$, con notifica m''. Quando m'' arriva è la prossima notifica di p_1 ($D[1] = 1 = 2-1$) e il monitor ha già consegnato la notifica di p_2 che p_1 conosceva ($D[2] = 1 \geq 1$): viene consegnata subito, e D diventa $[2,1]$.

Con DR3 si ha una soluzione completa di GPE con monitoraggio passivo in un sistema asincrono: i processi notificano gli eventi rilevanti, p_0 li consegna in ordine causale e osserva un run consistente, in cui Φ si può valutare in qualunque stato. La stessa regola funziona in ogni processo, non solo nel monitor: se tutti comunicano con *broadcast*, cioè inviando ogni messaggio a tutti, DR3 dà il *causal broadcast*, una primitiva molto usata.

Canali nascosti. La precedenza causale vede solo la comunicazione che passa per i canali del sistema (un **sistema chiuso**); un **canale nascosto** (*hidden channel*), esterno al sistema, le sfugge. Per esempio p_1 rileva la rottura di un tubo del vapore e lo notifica a p_3, che controlla il riscaldamento; poco dopo p_2, più a valle, rileva un calo di pressione e notifica anche quello. Per il sistema le due notifiche sono concorrenti: p_3 può vedere prima il calo di pressione, aumentare il calore e concludere a torto che il tubo si è rotto per il calore. Il legame tra le due cose passa per il tubo, fuori dal sistema; in casi così bisogna ordinare le osservazioni con il tempo reale.

Capitolo 5

Snapshot distribuiti

Nel capitolo 2 il monitor costruiva uno stato globale chiedendo a ogni processo il proprio stato, e il risultato poteva essere inconsistente. Un protocollo di **snapshot** (istantanea) fa la stessa cosa, ma garantisce uno stato globale consistente, senza fermare la computazione. Si parte da un protocollo che usa un orologio reale, lo si trasforma nel protocollo di Chandy e Lamport, che funziona in un sistema asincrono, e infine si vede che cosa garantisce lo stato costruito. In tutto il capitolo i canali sono FIFO.

5.1 Che cosa registra uno snapshot

Uno snapshot è una fotografia del sistema, e deve contenere due cose: lo stato locale di ogni processo e i messaggi che in quel momento sono in viaggio.

> **Definizione (stato di un canale).** Lo **stato del canale** da p_i a p_j, indicato con $\chi_{i,j}$ (con la lettera greca *chi*), è l'insieme dei messaggi in viaggio su quel canale: quelli che p_i ha inviato a p_j e che p_j non ha ancora ricevuto.

I canali su cui un processo riceve sono i suoi **canali entranti**, quelli su cui invia i suoi **canali uscenti**. Ogni processo registra il proprio stato locale e lo stato dei propri canali entranti: così ogni canale viene registrato una volta sola, da chi sta all'estremità di arrivo. Le difficoltà sono due: i processi non possono registrare tutti nello stesso istante, e i messaggi in viaggio non li vede nessuno.

5.2 Primo protocollo: un orologio reale

Supponiamo per ora che tutti i processi leggano lo stesso orologio reale RC, e che i ritardi dei messaggi e le velocità dei processi abbiano limiti noti. Allora la soluzione è ovvia: tutti registrano il proprio stato nello stesso istante, come in una foto di gruppo. Il monitor p_0 sceglie l'istante dello snapshot, t_{ss} (*time* dello *snapshot*), abbastanza avanti nel tempo perché il suo annuncio arrivi a tutti prima. Ogni messaggio porta come timestamp l'istante in cui è stato inviato.

Definizione (Protocollo 1, con orologio reale).

1. p_0 invia a tutti i processi il messaggio «take snapshot at t_{ss}».
2. Quando l'orologio segna t_{ss}, ogni processo p_i, prima di qualunque altro evento, registra il proprio stato locale σ_i, invia un messaggio vuoto su ogni canale uscente e comincia a registrare i messaggi che arrivano su ogni canale entrante.
3. La prima volta che riceve da p_j un messaggio con timestamp maggiore o uguale a t_{ss}, p_i smette di registrare quel canale: $\chi_{j,i}$ è l'insieme dei messaggi registrati.

Sul canale da p_j a p_i, all'istante t_{ss}, sono in viaggio i messaggi inviati prima di t_{ss} e ricevuti dopo. p_i registra proprio quelli: comincia a registrare in t_{ss} e si ferma al primo messaggio inviato da t_{ss} in poi, perché il canale è FIFO e a quel punto tutti i messaggi inviati prima sono già arrivati. I messaggi vuoti servono solo a far arrivare quel primo messaggio anche su un canale su cui p_j non avrebbe inviato più nulla.

Lo stato costruito è consistente, perché il sistema ci si è trovato davvero, nell'istante t_{ss}. Conta però capire che cosa lo garantisce. Il taglio dello snapshot contiene gli eventi avvenuti prima di t_{ss}. Se e è nel taglio ed e' è una sua causa, allora $RC(e') < RC(e) < t_{ss}$, dove $RC(e)$ è l'istante in cui avviene e, e la prima disuguaglianza è la clock condition: anche e' è nel taglio, che quindi contiene le cause di ogni suo evento.

5.3 Dall'orologio reale ai marker

Il Protocollo 1 usa dell'orologio reale soltanto la clock condition, come la regola DR1 (Sezione 3.3), e la clock condition vale anche per i clock logici. Da qui si arriva al protocollo di Chandy–Lamport in tre passi, riassunti nella Tabella 5.1.

1. Clock logici al posto dell'orologio (Protocollo 2). Le regole sono quelle del Protocollo 1, con il clock logico LC di ogni processo al posto dell'orologio reale. Cambia la scelta dell'istante: in un sistema asincrono p_0 non può calcolare un valore «abbastanza avanti». Si usa allora un valore ω (*omega*) così grande che nessun clock lo raggiunge con gli eventi normali della computazione. p_0 invia l'annuncio «take snapshot at ω», porta il proprio clock a ω, registra il proprio stato e invia i messaggi vuoti, con timestamp ω. Quando un altro processo riceve un messaggio con timestamp ω, il suo clock salta oltre ω: il processo registra il proprio stato subito prima di quella ricezione e invia a sua volta i messaggi vuoti. Così la registrazione si propaga a tutti. Resta un'ipotesi scomoda: perché un valore come ω esista, servono comunque limiti alle velocità e ai ritardi.

2. Niente annuncio. Nel Protocollo 2 un processo non fa nulla tra l'arrivo dell'annuncio e l'arrivo del primo messaggio vuoto, ed è quel messaggio a farlo registrare. L'annuncio quindi non serve: un processo registra quando riceve il primo

messaggio vuoto. Perché non si confonda con altri messaggi, gli si dà un contenuto riconoscibile, «take snapshot», e lo si chiama **marker** (segnalino).

3. Niente timestamp. Su ogni canale il marker è il primo messaggio che il mittente invia dopo essersi registrato, e il canale è FIFO. Quindi i messaggi che arrivano prima del marker sono stati inviati prima della registrazione del mittente, e quelli che arrivano dopo sono stati inviati dopo: il marker *separa* i due gruppi, e fa da solo il lavoro del timestamp. Spariscono così i timestamp, i clock logici e ω, e resta il protocollo di Chandy–Lamport, che non fa ipotesi oltre ai canali FIFO.

Tabella 5.1. I tre protocolli di snapshot a confronto.

	Protocollo 1	Protocollo 2	Chandy–Lamport
Quando p_i registra il proprio stato	quando l'orologio reale segna t_{ss}	quando il suo clock logico raggiunge ω	alla prima ricezione di un marker
Quando smette di registrare il canale da p_j	al primo messaggio di p_j con timestamp $\geq t_{ss}$	al primo messaggio di p_j con timestamp $\geq \omega$	quando arriva il marker di p_j
Ipotesi, oltre ai canali FIFO	orologio comune; ritardi e velocità limitati	un valore ω che nessun clock raggiunge da solo	nessuna

5.4 Il protocollo di Chandy–Lamport

Definizione (protocollo di Chandy–Lamport). p_0 avvia il protocollo inviando un marker a sé stesso. Quando un processo p_i riceve un marker dal canale di p_j:

- se è il *primo* marker che riceve, registra il proprio stato locale σ_i e invia un marker su ogni canale uscente, senza eventi della computazione in mezzo; poi registra vuoto il canale da cui è arrivato il marker ($\chi_{j,i} = \emptyset$) e comincia a registrare i messaggi che arrivano su ciascuno degli altri canali entranti;
- altrimenti, smette di registrare il canale da p_j: $\chi_{j,i}$ è l'insieme dei messaggi registrati su quel canale.

Indichiamo con e_i^* il momento in cui p_i riceve il primo marker e registra il proprio stato: divide gli eventi di p_i in quelli che vengono prima della registrazione e quelli che vengono dopo. Nelle figure è un quadratino.

Perché i canali si registrano così? $\chi_{j,i}$ deve contenere i messaggi che p_j ha inviato prima di registrarsi e che p_i riceve dopo essersi registrato. I primi sono quelli che arrivano prima del marker di p_j (passo 3 della Sezione 5.3), e tra questi p_i deve tenere quelli che arrivano dopo la propria registrazione. Quindi registra il canale da

quando si registra fino all'arrivo del marker di p_j; sul canale da cui arriva il primo marker questo intervallo è vuoto.

Un processo inoltra il marker solo alla prima ricezione, e la rete è fortemente connessa: il marker raggiunge tutti e attraversa ogni canale esattamente una volta. Quando p_i ha ricevuto il marker da tutti i canali entranti, ha registrato il proprio stato e quelli di tutti i suoi canali entranti, e per lui il protocollo è finito; resta solo da inviare i risultati a p_0. Il monitor ha quindi due compiti, avviare lo snapshot e raccoglierne i pezzi; il suo stato non fa parte dello snapshot, perché non partecipa alla computazione.

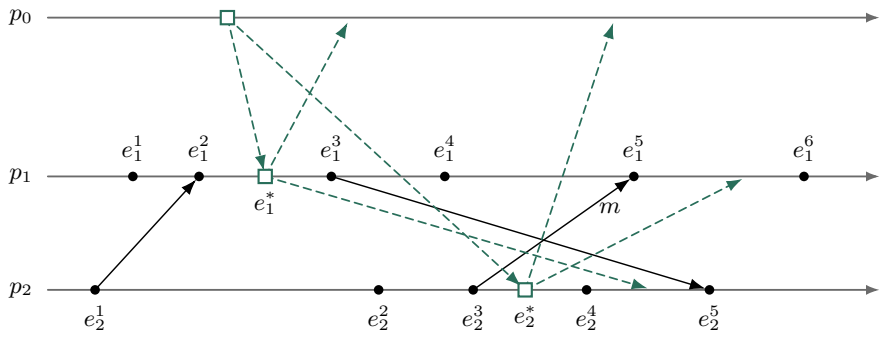


Figura 5.1. Il protocollo di Chandy–Lamport sulla computazione della Figura 2.3. Tratteggiati i marker; i quadratini segnano le registrazioni (e_1^* ed e_2^*).

Esempio (il protocollo passo passo). Nella Figura 5.1 il protocollo viene eseguito sulla computazione di due processi della Figura 2.3.

- p_0 registra il proprio stato e invia il marker a p_1 e a p_2.
- p_1 riceve il marker dopo e_1^2: registra σ_1^2, inoltra il marker a p_0 e a p_2, pone $\chi_{0,1} = \emptyset$ e comincia a registrare il canale da p_2.
- p_2 riceve il marker di p_0 solo dopo e_2^3: registra σ_2^3, inoltra il marker, pone $\chi_{0,2} = \emptyset$ e comincia a registrare il canale da p_1.
- Il messaggio m, che p_2 ha inviato in e_2^3, prima di registrarsi, arriva a p_1 prima del marker di p_2: viene registrato, e $\chi_{2,1} = \{m\}$.
- Il messaggio che p_1 ha inviato in e_1^3, dopo essersi registrato, arriva a p_2 dopo il marker di p_1: non viene registrato, e $\chi_{1,2} = \emptyset$.

Lo snapshot è lo stato globale Σ^{23} (due eventi di p_1, tre di p_2), con m in viaggio da p_2 a p_1.

5.5 Perché lo stato è consistente

Un taglio è inconsistente solo se contiene un messaggio «dal futuro», ricevuto dentro il taglio ma inviato fuori (capitolo 2). Con il protocollo di Chandy–Lamport non succede. Un messaggio che p_j invia dopo essersi registrato, cioè fuori dal taglio, viaggia dietro al marker di p_j: quando arriva, il destinatario ha già ricevuto quel

marker e si è già registrato, quindi anche la ricezione è fuori dal taglio. Qui servono i canali FIFO: se un messaggio potesse superare il marker, il destinatario potrebbe riceverlo prima di registrarsi, e lo snapshot conterrebbe una ricezione senza il suo invio.

5.6 Proprietà dello snapshot

Lo stato costruito è consistente, ma il sistema potrebbe non esserci mai passato. Nell'esempio, quando p_1 si registra p_2 ha eseguito un solo evento, e quando p_2 si registra p_1 è già arrivato a e_1^4: nel run reale, il cammino blu della Figura 5.2, lo stato Σ^{23} non compare. Lo snapshot però non è uno stato consistente qualunque. Siano:

- Σ^a (a come avvio) lo stato del sistema quando il protocollo parte;
- Σ^s (s come snapshot) lo stato costruito dal protocollo;
- Σ^f (f come fine) lo stato del sistema quando il protocollo termina, cioè quando arriva l'ultimo marker.

Allora

$$\Sigma^a \rightsquigarrow \Sigma^s \rightsquigarrow \Sigma^f,$$

cioè esiste un run consistente che passa, nell'ordine, per l'avvio, per lo snapshot e per la fine: se non è quello reale, è uno che il sistema avrebbe potuto seguire.

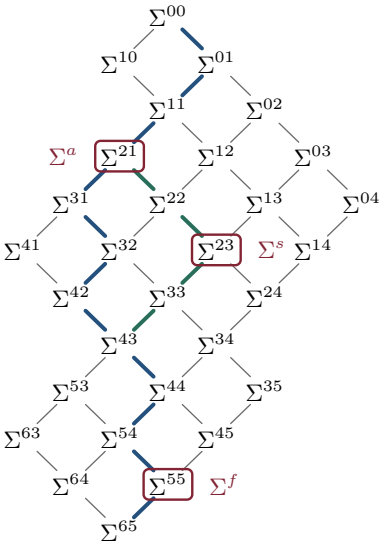


Figura 5.2. Lo snapshot nel reticolo della Figura 2.3. Il run reale (blu) passa per l'avvio $\Sigma^a = \Sigma^{21}$ e la fine $\Sigma^f = \Sigma^{55}$, ma non per lo stato costruito $\Sigma^s = \Sigma^{23}$. Il run riordinato (verde) se ne discosta solo tra Σ^{21} e Σ^{43}, e passa per tutti e tre.

Perché. Un evento di p_i è **pre-registrazione** (*prerecording*) se viene prima di e_i^*, **post-registrazione** (*postrecording*) se viene dopo. Σ^s è lo stato in cui si arriva eseguendo tutti gli eventi pre-registrazione e nessun altro. Per l'argomento della Sezione 5.5, un evento post-registrazione non può precedere causalmente un evento

pre-registrazione. Quindi, nel run reale, si possono spostare tutti gli eventi pre-registrazione davanti a quelli post-registrazione senza mai mettere un effetto prima della sua causa, e il run che si ottiene è consistente. Passa per Σ^a, perché gli eventi avvenuti prima dell'avvio sono tutti pre-registrazione e restano in testa; per Σ^s, subito dopo l'ultimo evento pre-registrazione; per Σ^f, perché gli eventi avvenuti dopo la fine sono tutti post-registrazione e restano in coda.

Nell'esempio gli eventi pre-registrazione sono e_1^1, e_1^2 ed e_2^1, e_2^2, e_2^3. Nel run reale gli eventi post-registrazione e_1^3 ed e_1^4 vengono prima di alcuni eventi pre-registrazione di p_2; anticipando questi ultimi si ottiene il run

$$e_2^1\, e_1^1\, e_1^2\, e_2^2\, e_2^3 \mid e_1^3\, e_1^4\, e_2^4\, e_1^5\, e_2^5\, e_1^6,$$

dove la sbarra separa i due gruppi. È il cammino verde della Figura 5.2, e passa per $\Sigma^a = \Sigma^{21}$, per $\Sigma^s = \Sigma^{23}$ e per $\Sigma^f = \Sigma^{55}$.

A che cosa serve. Lo snapshot descrive uno stato che forse non è mai esistito, e che comunque è già passato. Serve per le proprietà **stabili**, quelle che, una volta diventate vere, restano vere, come il deadlock o la terminazione. Per una proprietà così, se vale in Σ^s vale anche in Σ^f, cioè adesso; se non vale in Σ^s, non valeva nemmeno in Σ^a, quando il protocollo è partito. Le proprietà stabili si riprenderanno più avanti.

Capitolo 6

Guasti e commit atomico

Finora i processi non si guastavano e ogni messaggio arrivava, e il problema era osservare il sistema. Da qui in poi siti e collegamenti si possono guastare, e il problema diventa far prendere a un gruppo di processi *la stessa decisione*. Il caso tipico è la conclusione di una transazione distribuita, che va confermata in tutti i siti in cui ha lavorato oppure annullata in tutti. Questo capitolo descrive il problema, i guasti possibili e i limiti di ogni soluzione; il capitolo 7 presenta il protocollo più usato. Il contesto è quello delle basi di dati, ma le idee valgono per qualunque sistema distribuito.

6.1 Transazioni distribuite

Una **transazione** è una sequenza di letture e scritture da eseguire come un'operazione sola: o hanno effetto tutte, o nessuna. Termina con il **Commit** (conferma), che rende definitivi i suoi effetti, oppure con l'**Abort** (annullamento), che li cancella come se non fosse mai stata eseguita.

In una base di dati distribuita i dati stanno su più **siti**, cioè su computer collegati da una rete, e supponiamo che ogni dato stia in un solo sito. Una transazione distribuita T nasce in un sito, il suo *home site*, dove il *transaction manager* (TM) inoltra ogni lettura o scrittura del dato x al sito in cui sta x: per letture e scritture non cambia niente rispetto a una base di dati centralizzata. Il Commit e l'Abort, invece, riguardano tutti i siti in cui T ha lavorato, e ognuno li deve eseguire. Ne nascono due difficoltà:

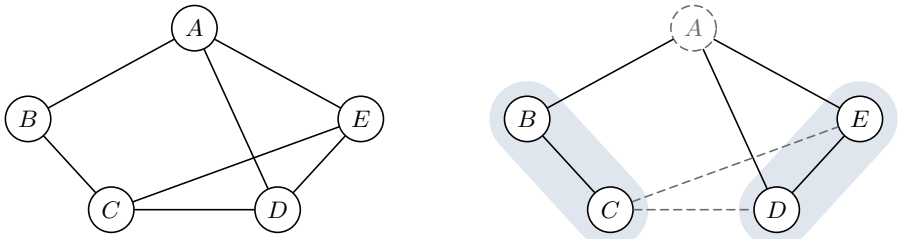
- *non basta inviare il Commit*: T è confermata quando ogni sito lo esegue, e un sito potrebbe rifiutarsi, per esempio perché il suo controllo della concorrenza ha annullato T; allora T va annullata ovunque;
- *i guasti sono parziali*: in un sistema centralizzato un guasto ferma tutto, mentre in un sistema distribuito alcuni siti possono funzionare e altri no.

Il problema è la **terminazione consistente**: la conclusione di T deve essere la stessa in tutti i siti coinvolti, anche quando alcuni si guastano. Un algoritmo che la garantisce è un **protocollo di commit atomico** (*atomic commitment protocol*, ACP), e lo si vuole resistente al maggior numero possibile di guasti.

Esempio (bonifico tra due banche). T sposta 100 euro dal conto x, nel sito A, al conto y, nel sito B. Se A esegue il Commit e B l'Abort, per esempio perché B si è guastato prima di salvare la scrittura, i 100 euro spariscono; nel caso opposto vengono creati dal nulla. Nella Sezione 1.1 bisognava osservare il bonifico senza sbagliare; qui i due siti devono prendere la stessa decisione.

6.2 Il modello dei guasti

Un sistema distribuito è fatto di **siti**, che elaborano, e di **collegamenti** (*link*) bidirezionali, che trasportano i messaggi: è un grafo connesso, come nella Figura 6.1(a). L'instradamento dei messaggi è un servizio della rete, e non ci interessa.



(a) la rete (b) una partizione

Figura 6.1. (a) Una rete di cinque siti. (b) Una partizione della stessa rete: il sito A è guasto e i collegamenti (C, D) e (C, E) sono interrotti (tratteggiati), quindi i siti operativi si dividono in due componenti, $\{B, C\}$ e $\{D, E\}$ (in azzurro).

6.2.1 Guasti dei siti

Un sito che si guasta si ferma di colpo e perde il contenuto della **memoria volatile** (la memoria centrale); quello della **memoria stabile**, come i dischi, resta. Quando riparte, una procedura di recupero lo riporta in uno stato corretto. Si assume che un sito sia sempre **operativo**, e allora funzioni correttamente, oppure **guasto** (*down*), e allora non faccia niente: non esegue mai azioni sbagliate. È il modello **fail-stop**, un'idealizzazione a cui ci si avvicina con test accurati e componenti ridondanti.

Si ha un **guasto parziale** quando alcuni siti sono operativi e altri no, un **guasto totale** quando sono guasti tutti. I guasti parziali sono insidiosi perché un sito operativo può non sapere che cosa avesse deciso un sito guasto, e questo può costringerlo ad aspettare (Sezione 6.4).

6.2.2 Guasti di comunicazione

Messaggi alterati o persi e collegamenti interrotti per poco li gestisce la rete, con codici che rilevano gli errori, ritrasmissioni e percorsi alternativi. Non c'è rimedio, però, se tutti i percorsi tra due siti passano per un sito guasto o per un collegamento interrotto: è una **partizione della rete** (*network partition*). I siti operativi si dividono in **componenti**: due siti della stessa componente comunicano, due di componenti diverse no. Nella Figura 6.1(b) il guasto di A e dei collegamenti (C, D) e

(C, E) produce le componenti $\{B, C\}$ e $\{D, E\}$, che si riuniscono appena A riparte o un collegamento torna a funzionare. In generale si ha un **guasto di comunicazione** quando due siti non guasti non riescono a comunicare; quando invece comunicano, i messaggi arrivano integri.

Un messaggio che non si può consegnare, perché il destinatario è guasto o sta in un'altra componente, si perde, e nessuno lo viene a sapere: farlo conservare dalla rete sposterebbe soltanto il problema, e un avviso di mancata consegna non sarebbe affidabile, perché la rete non distingue un destinatario che non ha ricevuto da uno che ha ricevuto e si è guastato subito dopo.

6.2.3 Rilevare i guasti con i timeout

Per chi aspetta, un sito guasto e una partizione sono indistinguibili: in entrambi i casi non arrivano risposte. Per accorgersene si usa un **timeout**: A invia un messaggio a B e ne aspetta la risposta per un **periodo di timeout**, che deve coprire l'andata, l'elaborazione e il ritorno; se il periodo scade, A conclude di non poter comunicare con B. Stabilire il periodo è difficile, perché dipende dal carico, dalla rete e dagli orologi, e in pratica si sceglie un valore che funziona quasi sempre.

Se il periodo è troppo corto, A può credere di non comunicare con B anche quando potrebbe: è un ***timeout failure*** (o *performance failure*), che conta come un guasto di comunicazione ma crea situazioni che una partizione non crea, come A che crede di comunicare con B mentre B crede il contrario. Usare un timeout vuol dire supporre un limite ai ritardi, come nei sistemi sincroni della Sezione 1.2, ma quel limite può essere sbagliato: per ogni protocollo bisogna quindi dire se tollera solo i guasti dei siti o anche quelli di comunicazione.

6.3 Il problema del commit atomico

Prima di far eseguire il Commit, il TM deve sapere che ogni sito è pronto a eseguirlo. Un sito lo è se può garantire il Commit di T anche dopo un proprio guasto, cioè se:

- in quel sito T non ha letto valori scritti da transazioni non ancora confermate: se una di queste venisse annullata, andrebbe annullata anche T;
- tutti i valori che T ha scritto in quel sito sono già in memoria stabile. In un sito in cui T ha solo letto, la condizione è vera in automatico.

Dare il consenso è quindi una promessa. L'idea di soluzione è semplice: si fa eseguire il Commit solo dopo il consenso di tutti i siti, ed è il Two-Phase Commit del capitolo 7. La difficoltà sta nei guasti: siti che si fermano a metà, messaggi che si perdono.

Per concentrarsi sul commit, per ogni transazione T immaginiamo un processo in ogni sito in cui T ha lavorato. Quello dell'*home site* è il **coordinatore**, gli altri sono i **partecipanti**: il coordinatore conosce i partecipanti, mentre i partecipanti conoscono il coordinatore ma non necessariamente si conoscono tra loro. (In pratica gli stessi compiti li svolgono pochi processi per sito, condivisi da molte transazioni.) Ogni sito ha anche un **DT log** (*distributed transaction log*), un registro in memoria stabile in cui i processi annotano ciò che devono ricordare: dopo un guasto lo rileggono per sapere a che punto erano.

Ogni processo esprime un **voto**, Yes oppure No, e raggiunge una **decisione**, Commit oppure Abort. Vota Yes solo se il suo sito è pronto a confermare T; quando decide, il suo sito esegue il Commit o l'Abort di T per conto proprio.

Definizione (protocollo di commit atomico). Ogni processo può esprimere esattamente uno dei due voti, Yes o No, e può raggiungere esattamente una delle due decisioni, Commit o Abort. Un **protocollo di commit atomico** (ACP) è un algoritmo con cui i processi raggiungono le decisioni in modo che valgano cinque condizioni:

AC1 tutti i processi che raggiungono una decisione raggiungono la stessa;

AC2 un processo non può cambiare la propria decisione dopo averla raggiunta;

AC3 la decisione Commit si può raggiungere solo se tutti i processi hanno votato Yes;

AC4 se non ci sono guasti e tutti i processi hanno votato Yes, la decisione è Commit;

AC5 in ogni esecuzione in cui capitano solo guasti che l'algoritmo è progettato per tollerare, se a un certo punto tutti i guasti vengono riparati e per un tempo abbastanza lungo non se ne verificano di nuovi, prima o poi tutti i processi raggiungono una decisione.

La definizione merita tre commenti.

- AC1 non chiede che *tutti* decidano: un processo può guastarsi e non ripartire più, e come si vedrà nella Sezione 6.4 non si può pretendere nemmeno che decidano tutti i processi operativi. AC5 chiede invece che tutti decidano una volta riparati i guasti, ed esclude i protocolli che, dopo un guasto qualunque, lasciano i processi indecisi per sempre.
- AC4 esclude i protocolli banali che decidono sempre Abort, che soddisferebbero tutte le altre condizioni.
- AC4 è solo un inverso parziale di AC3: con dei guasti si può decidere Abort anche se tutti hanno votato Yes (Sezione 7.2).

6.4 Incertezza e blocco

Da AC3 segue che un processo che non ha ancora votato Yes può decidere Abort da solo, in qualunque momento: senza il suo Yes nessuno potrà decidere Commit. Dopo aver votato Yes, invece, non può più decidere da solo, perché gli altri potrebbero aver deciso Commit oppure Abort.

Definizione (periodo di incertezza). Il **periodo di incertezza** (*uncertainty period*) di un processo va dal momento in cui vota Yes al momento in cui riceve abbastanza informazioni per sapere quale sarà la decisione. Durante questo periodo il processo è **incerto**: non sa se la decisione sarà Commit o Abort, e

non può decidere Abort da solo.

Se un guasto colpisce un processo incerto, ci sono due scenari.

Scenario I: il processo resta isolato. Un guasto di comunicazione separa il processo incerto p da tutti gli altri, e p non può decidere finché il guasto non viene riparato. Un processo che deve aspettare la riparazione di guasti per andare avanti è **bloccato** (*blocked*). Incerto e bloccato non sono la stessa cosa: un processo incerto non conosce la decisione, ma può scoprirla chiedendo agli altri; uno bloccato è incerto e non può nemmeno scoprirla, finché un guasto non viene riparato. Il blocco può durare un tempo arbitrario, e intanto T resta aperta nel sito di p e tiene occupate risorse, come i *lock* con cui impedisce alle altre transazioni di toccare i suoi dati.

Scenario II: il processo si guasta. p si guasta mentre è incerto, e quando riparte deve chiedere agli altri che cosa è stato deciso. La capacità di decidere da soli quando si riparte dopo un guasto si chiama **recupero indipendente** (*independent recovery*). Rende il recupero più semplice ed economico, e senza di essa un guasto totale porta al blocco: il primo processo a ripartire, se è incerto, non trova nessuno a cui chiedere.

Il periodo di incertezza non si può eliminare: servirebbe che ogni processo votasse e conoscesse i voti di tutti gli altri nello stesso istante. Ne seguono due limiti, che valgono per qualunque protocollo.

Due limiti di ogni protocollo di commit atomico.

- Se sono possibili guasti di comunicazione o guasti totali, *ogni* ACP può far bloccare dei processi: è lo scenario I, o lo scenario II con un guasto totale.
- Nessun ACP può garantire il recupero indipendente ai processi che si guastano: è lo scenario II.

Il primo limite lascia aperta una possibilità: se si possono guastare solo i siti, e mai tutti insieme, esiste un ACP che non blocca mai, il *Three-Phase Commit*, che qui non si tratta.

Capitolo 7

Il protocollo Two-Phase Commit

Il **Two-Phase Commit** (2PC, commit in due fasi) è il protocollo di commit atomico più semplice e più usato. Lo si vede prima senza guasti, poi con i timeout, il recupero dopo un guasto e i costi. Indichiamo con c il coordinatore e con $p_1, \ldots, p_n$ i partecipanti, quindi i processi sono $n+1$; p e q sono partecipanti qualunque. Tre scritture distinguono tre cose con lo stesso nome: il voto Yes è la scelta di un processo, YES (in maiuscoletto) il messaggio che la comunica, `yes` (in carattere macchina) il record che il processo scrive nel DT log. Lo stesso vale per le decisioni Commit e Abort, i messaggi COMMIT e ABORT e i record `commit` e `abort`.

7.1 Il protocollo

Definizione (Two-Phase Commit, senza guasti).

1. Il coordinatore invia a tutti i partecipanti un messaggio VOTE-REQ (*vote request*, richiesta di voto).
2. Quando riceve VOTE-REQ, ogni partecipante risponde con il proprio voto, YES oppure NO. Se vota No, decide subito Abort e si ferma.
3. Il coordinatore raccoglie i voti. Se sono tutti YES e anche il suo voto è Yes (il coordinatore vota per il proprio sito), decide Commit e invia COMMIT a tutti i partecipanti; altrimenti decide Abort e invia ABORT a chi ha votato Yes. Poi si ferma.
4. Ogni partecipante che ha votato Yes aspetta COMMIT o ABORT, decide di conseguenza e si ferma.

I passi 1 e 2 formano la **fase di voto**, i passi 3 e 4 la **fase di decisione**.

Esempio (due esecuzioni senza guasti). Nella Figura 7.1(a) tutti votano Yes: ogni partecipante è incerto dall'invio di YES all'arrivo di COMMIT, e c decide Commit appena ha tutti i voti. Nella Figura 7.1(b) p_2 vota No: decide subito Abort, senza mai essere incerto, e c invia ABORT solo a p_1 e p_3, gli unici che non hanno ancora deciso.

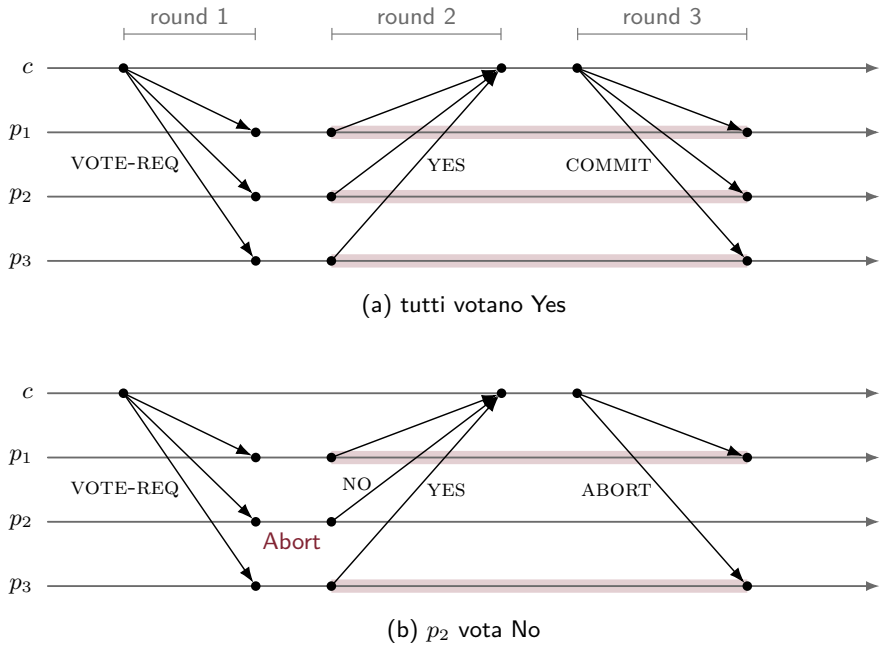


Figura 7.1. Due esecuzioni del 2PC senza guasti, con il coordinatore c e i partecipanti p_1, p_2, p_3. Le fasce rosse sono i periodi di incertezza dei partecipanti. In (a) sono segnati anche i tre round del protocollo (Sezione 7.4).

Il coordinatore, invece, non è mai incerto: decide nel momento in cui vota, quando conosce già tutti i voti. Senza guasti il 2PC soddisfa le condizioni da AC1 ad AC4: le decisioni vengono tutte dal coordinatore, che decide Commit solo se tutti i voti, compreso il suo, sono Yes, mentre chi vota No decide Abort come il coordinatore; ognuno decide una volta sola; e se non ci sono guasti e tutti votano Yes, la decisione è Commit. Non soddisfa invece AC5, per due motivi:

- un processo che aspetta un messaggio che non arriverà mai aspetterebbe per sempre: servono azioni da eseguire allo scadere di un timeout (Sezione 7.2);
- un processo che riparte dopo un guasto deve decidere in modo coerente con gli altri, quindi deve aver salvato qualcosa nel DT log (Sezione 7.3).

7.2 Timeout e protocollo di terminazione

Quando l'attesa di un messaggio supera il timeout, il processo esegue un'**azione di timeout** (*timeout action*). Nel 2PC si aspetta in tre punti, all'inizio dei passi 2, 3 e 4 (Tabella 7.1).

Nei primi due casi chi va in timeout decide Abort da solo. Il secondo mostra perché si può decidere Abort anche se tutti hanno votato Yes: basta che un YES arrivi al coordinatore in ritardo, o non arrivi per una partizione.

Nel terzo caso il partecipante p è incerto e deve consultare altri processi: è il **protocollo di terminazione** (*termination protocol*), cioè ciò che fa un processo incerto quando un messaggio atteso non arriva. Il più semplice è aspettare il

Tabella 7.1. Le azioni di timeout del 2PC.

Chi aspetta, e che cosa	Allo scadere del timeout	Perché è sicuro
un partecipante aspetta VOTE-REQ (passo 2)	decide Abort e si ferma	non ha ancora votato, quindi può decidere Abort da solo
il coordinatore aspetta i voti (passo 3)	decide Abort e invia ABORT ai partecipanti da cui ha ricevuto YES	non ha ancora deciso, e senza la sua decisione nessun partecipante può aver deciso Commit
un partecipante che ha votato Yes aspetta la decisione (passo 4)	avvia il protocollo di terminazione	è incerto: non può decidere da solo e deve chiedere agli altri

coordinatore: prima o poi torna raggiungibile, e conosce sempre la decisione perché non è mai incerto. AC5 è rispettata, ma p può restare bloccato senza bisogno.

Esempio (un blocco inutile). Il coordinatore invia la decisione al partecipante q e si guasta prima di inviarla a p. p è incerto, ma q no: se p potesse chiedere a q, non dovrebbe aspettare che il coordinatore riparta.

Per chiedersi la decisione, i partecipanti devono conoscersi: basta che il coordinatore aggiunga a VOTE-REQ l'elenco dei partecipanti. Prima di riceverlo nessuno ne ha bisogno, perché chi va in timeout aspettando VOTE-REQ decide Abort da solo.

Definizione (protocollo di terminazione cooperativo). Un partecipante p che va in timeout mentre è incerto diventa l'**iniziatore** (*initiator*) del protocollo; i processi a cui si rivolge sono i **rispondenti** (*responder*).

- *Iniziatore.* p invia un messaggio DECISION-REQ (*decision request*, richiesta di decisione) a tutti gli altri processi, coordinatore compreso, e aspetta da uno qualunque di loro una risposta, COMMIT o ABORT. Quando la riceve, decide di conseguenza. Se scade il timeout senza risposte, ricomincia da capo: è bloccato.
- *Rispondente.* Un processo q che riceve DECISION-REQ da p si trova in uno di tre casi:
  1. ha già deciso Commit o Abort: invia a p COMMIT o ABORT;
  2. non ha ancora votato: decide Abort, come può fare da solo, e invia ABORT a p;
  3. ha votato Yes ma non ha ancora deciso: è incerto anche lui, quindi non può aiutare p, e non risponde.

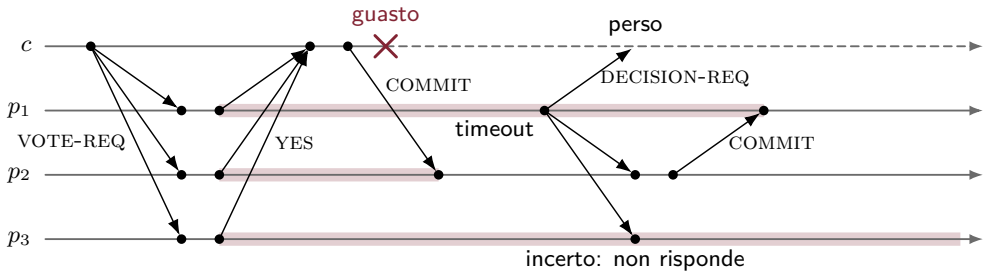


Figura 7.2. Il protocollo di terminazione cooperativo. Il coordinatore c invia COMMIT solo a p_2 e si guasta; p_1 va in timeout e chiede la decisione a tutti, e la riceve da p_2. Le fasce rosse sono i periodi di incertezza.

Esempio (terminazione cooperativa). Nella Figura 7.2 tutti votano Yes; c decide Commit, invia COMMIT solo a p_2 e si guasta. Quando il suo timeout scade, p_1 invia DECISION-REQ a tutti: la richiesta a c si perde, p_3 è incerto e non risponde (caso 3), p_2 risponde COMMIT (caso 1). Così p_1 decide Commit senza aspettare c, e lo stesso farà p_3 al suo timeout.

p resta bloccato solo se tutti i processi che riesce a raggiungere sono incerti, e lo resta finché la riparazione dei guasti non gli fa raggiungere un processo nel caso 1 o 2. Un processo così esiste sempre, il coordinatore, quindi AC5 è rispettata. I blocchi diventano meno probabili, ma non spariscono, come vuole il primo limite della Sezione 6.4: il 2PC si può bloccare anche quando si guastano soltanto i siti.

7.3 Recupero e DT log

Un processo p che riparte dopo un guasto deve arrivare a una decisione coerente con quella degli altri, subito o dopo la riparazione degli altri guasti (AC5). Se ricorda a che punto era:

- se si è guastato prima di inviare YES, decide Abort da solo;
- se aveva già deciso, non deve fare niente;
- se era incerto, non può decidere da solo (scenario II della Sezione 6.4): è come se fosse andato in timeout aspettando la decisione, e avvia il protocollo di terminazione, con il rischio di restare bloccato.

7.3.1 Che cosa scrivere nel DT log

Per ricordare a che punto era, ogni processo scrive dei record nel DT log del proprio sito.

1. Il coordinatore, quando invia i VOTE-REQ, scrive `start-2PC`, con i nomi dei partecipanti, prima o dopo l'invio.
2. Un partecipante che vota Yes scrive `yes`, con i nomi del coordinatore e degli altri partecipanti, *prima* di inviare YES. Se vota No, scrive `abort`, prima o dopo l'invio di NO.

3. Il coordinatore scrive `commit` *prima* di inviare COMMIT.
4. Il coordinatore scrive `abort` quando invia ABORT, prima o dopo l'invio.
5. Un partecipante, ricevuta la decisione, scrive `commit` o `abort`.

Scrivere `commit` o `abort` è l'atto con cui un processo decide; da quel momento il suo sito può eseguire il Commit o l'Abort di T (Figura 7.3).

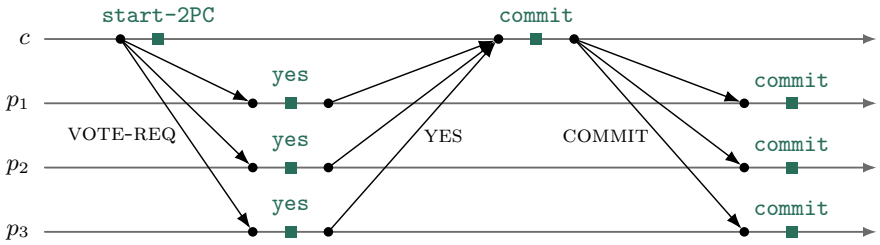


Figura 7.3. I record scritti nel DT log durante l'esecuzione della Figura 7.1(a): ogni quadratino verde è una scrittura. I partecipanti scrivono `yes` prima di inviare YES, il coordinatore scrive `commit` prima di inviare COMMIT.

Quando un sito riparte, dai record di ogni transazione ricava che cosa fare (Tabella 7.2).

Tabella 7.2. Che cosa fa un processo che riparte, in base ai record che trova nel DT log.

Record nel DT log	Situazione al momento del guasto	Che cosa fa
`start-2PC` e `commit` o `abort`	era il coordinatore, e aveva già deciso	niente: la decisione è quella del record
`start-2PC`, ma né `commit` né `abort`	era il coordinatore, e non aveva ancora deciso	decide Abort, scrivendo `abort`
niente `start-2PC`, ma `commit` o `abort`	era un partecipante, e aveva già deciso	niente
né `start-2PC` né `yes`	era un partecipante, e non aveva votato Yes: non aveva ancora votato, oppure aveva votato No senza aver ancora scritto `abort`	decide Abort, scrivendo `abort`
`yes`, ma né `commit` né `abort`	era un partecipante incerto	avvia il protocollo di terminazione, con i nomi dei processi presi dal record `yes`

Due record devono precedere il messaggio corrispondente, e la tabella mostra perché.

- *`commit` prima di* COMMIT. Altrimenti il coordinatore potrebbe inviare COMMIT e guastarsi prima di scrivere il record: ripartendo con il solo `start-2PC`

deciderebbe Abort, mentre i partecipanti hanno deciso Commit, e AC1 sarebbe violata.

- `yes` *prima di* YES. Altrimenti un partecipante potrebbe inviare YES e guastarsi prima di scrivere il record: ripartendo senza `yes` deciderebbe Abort, mentre il coordinatore potrebbe aver deciso Commit.

Per gli altri record l'ordine è libero. Chi riparte senza aver deciso e non trova un record che lo impegni decide Abort, quindi un record che porta ad Abort può mancare senza danni; uno che impegna verso il Commit, invece, deve essere in memoria stabile prima che gli altri ci possano contare.

7.3.2 Il protocollo completo

Mettendo insieme i passi della Sezione 7.1, le azioni di timeout e le regole del DT log si ottiene il protocollo completo.

Definizione (Two-Phase Commit, protocollo completo). *Coordinatore.*

1. Invia VOTE-REQ a tutti i partecipanti, con l'elenco dei partecipanti, e scrive `start-2PC` nel DT log.
2. Aspetta i voti di tutti i partecipanti. Se scade il timeout, scrive `abort`, invia ABORT ai partecipanti da cui ha ricevuto YES e termina.
3. Se tutti i voti sono YES e anche il suo voto è Yes, scrive `commit` e *poi* invia COMMIT a tutti i partecipanti. Altrimenti scrive `abort` e invia ABORT ai partecipanti da cui ha ricevuto YES.

Partecipante.

1. Aspetta VOTE-REQ dal coordinatore. Se scade il timeout, scrive `abort` e termina.
2. Se vota Yes, scrive `yes`, *poi* invia YES al coordinatore e aspetta COMMIT o ABORT. Se scade il timeout, avvia il protocollo di terminazione cooperativo; altrimenti scrive `commit` o `abort`, secondo il messaggio ricevuto.
3. Se vota No, scrive `abort` e invia NO al coordinatore.

Anche nel protocollo di terminazione decidere vuol dire scrivere `commit` o `abort`.

7.3.3 Più transazioni e ripresa di un sito

Il DT log di un sito contiene i record di molte transazioni, quindi ogni record porta il nome della sua. Per cancellare i record che non servono più (*garbage collection*) valgono due regole:

GC1 un sito non può cancellare i record di T dal proprio DT log prima di aver eseguito il Commit o l'Abort di T;

GC2 almeno un sito deve conservare i record di T finché non sa che il Commit o l'Abort di T è stato eseguito in tutti gli altri siti in cui T ha lavorato.

Senza GC2 un processo che riparte incerto potrebbe non trovare più nessuno che conosca la decisione, e AC5 sarebbe violata. GC1 si rispetta con le sole informazioni locali; GC2 richiede che ogni sito confermi di aver eseguito l'esito ai siti che conservano i record. Al limite li conserva solo il coordinatore, oppure tutti i siti.

Quando riparte, un sito deve completare il protocollo per le transazioni ancora indecise, ma non conviene che aspetti quelle bloccate prima di tornare a lavorare. Decide quelle che può, riprende a servire le altre transazioni e protegge i dati di quelle bloccate rimettendo i loro *lock* in scrittura. Quelli in lettura non servono più: una transazione bloccata ha finito di leggere, e conta solo che nessuno legga o sovrascriva i valori che ha scritto, che potrebbero ancora essere annullati. Poiché le tabelle dei *lock* stanno in memoria volatile, i *lock* in scrittura di T si annotano nel record `yes`, a meno che il sito non li possa ricavare dal proprio log.

7.4 Valutazione del 2PC

Un ACP si valuta secondo quattro criteri:

- **resilienza**: quali guasti tollera?
- **blocco**: i processi si possono bloccare, e in quali condizioni?
- **complessità in tempo**: quanto ci vuole per arrivare alla decisione?
- **complessità in messaggi**: quanti messaggi servono per arrivare alla decisione?

I primi due misurano l'affidabilità, gli altri due l'efficienza. Migliorare l'una costa nell'altra, e in ogni caso conviene ottimizzare il caso senza guasti, che si spera sia la normalità.

Round. La complessità in tempo si misura in **round**: un round è il tempo massimo che un messaggio impiega ad arrivare, lo stesso limite su cui si reggono i timeout. Due messaggi stanno in round diversi se e solo se uno non può partire prima che l'altro sia arrivato: nel 2PC un YES e un COMMIT stanno in round diversi, mentre tutti i VOTE-REQ stanno nello stesso round, e così tutti i COMMIT. Per contare i round si immagina che i messaggi di un round partano insieme e arrivino insieme, come nella Figura 7.1(a). Si trascurano il tempo di elaborazione, le scritture in memoria stabile e il costo di preparare le copie di un *broadcast* (l'invio dello stesso messaggio a molti destinatari), che a volte contano. La complessità in messaggi si misura contando i messaggi, che qui sono tutti brevi.

Resilienza. Il 2PC tollera sia i guasti dei siti sia quelli di comunicazione, partizioni o *timeout failure* che siano, perché le azioni di timeout della Tabella 7.1 sono sicure qualunque sia la causa del timeout.

> **Timeout sbagliati.** Le condizioni da AC1 ad AC4 non dipendono mai dalla durata del timeout: un timeout troppo corto può al più far decidere Abort quando si poteva confermare, o avviare il protocollo di terminazione quando non serviva. I timeout servono solo a non aspettare per sempre (AC5), e per questo le decisioni del 2PC restano coerenti anche in un sistema asincrono, dove

> nessun periodo di timeout è sicuro (Sezione 1.2).

Blocco. Un processo si blocca se va in timeout mentre è incerto e riesce a raggiungere solo processi incerti; succede anche quando si guastano soltanto i siti.

Complessità in tempo. Senza guasti servono tre round: i VOTE-REQ, i voti, la decisione. Il protocollo di terminazione ne aggiunge al più due, la richiesta e la risposta, anche se lo avviano più partecipanti, perché i loro round si sovrappongono. Quindi tutti i processi né guasti né bloccati decidono entro cinque round, *qualunque sia il numero dei guasti*; i processi bloccati, che possono aspettare senza limite, si escludono dal conto.

Complessità in messaggi. Ricordiamo che n è il numero dei partecipanti. In ognuno dei tre round partono al più n messaggi (n VOTE-REQ, n voti, al più n decisioni), quindi senza guasti il 2PC usa al più $3n$ messaggi.

Il protocollo di terminazione lo avviano i partecipanti che hanno votato Yes ma non hanno ricevuto la decisione; sia m il loro numero, con $0 \leq m \leq n$. Nel caso peggiore:

- *richieste*: ognuno degli m iniziatori invia DECISION-REQ agli altri n processi, in tutto nm messaggi;
- *risposte*: rispondono solo i processi non incerti. Al primo iniziatore possono rispondere gli $n - m$ partecipanti che non hanno avviato il protocollo e il coordinatore, cioè $n - m + 1$ processi. Ricevuta la risposta, il primo iniziatore ha deciso e può rispondere anche lui, quindi al secondo possono rispondere $n - m + 2$ processi, e in generale all'i-esimo iniziatore $n - m + i$.

Le risposte sono quindi al più

$$\sum_{i=1}^{m}(n - m + i) = m(n - m) + (1 + 2 + \cdots + m) = m(n - m) + \frac{m(m+1)}{2},$$

dove il primo addendo conta le $n - m$ risposte dei partecipanti non iniziatori, che arrivano a ognuno degli m iniziatori, e il secondo le i risposte che l'i-esimo riceve dal coordinatore e dai primi $i - 1$ iniziatori. Con le nm richieste, il protocollo di terminazione usa al più

$$nm + m(n - m) + \frac{m(m+1)}{2} = 2nm - \frac{m^2}{2} + \frac{m}{2}$$

messaggi. La quantità cresce con m, quindi è massima per $m = n$, cioè quando tutti i partecipanti vanno in timeout mentre sono incerti, e vale allora $n(3n + 1)/2$. In tutto, nel caso peggiore, il 2PC usa

$$3n + \frac{n(3n+1)}{2} = \frac{n(3n+7)}{2}$$

messaggi.

Esempio (tre partecipanti). Con $n = 3$, senza guasti servono al più $3 \cdot 3 = 9$ messaggi. Nel caso peggiore tutti e tre i partecipanti avviano il protocollo di terminazione ($m = 3$) e inviano $3 \cdot 3 = 9$ richieste. Alla prima può rispondere solo il coordinatore ($n - m + 1 = 1$), alla seconda anche il primo iniziatore, che ormai ha deciso (2), alla terza anche il secondo (3): le risposte sono $1+2+3 = 6$, e il protocollo di terminazione usa $9 + 6 = 15 = 3 \cdot (3 \cdot 3 + 1)/2$ messaggi. In tutto sono $9 + 15 = 24 = 3 \cdot (3 \cdot 3 + 7)/2$.